

LATEC: LEARNED AXIAL TENSOR COMPRESSION FOR HIGH-DIMENSIONAL SCIENTIFIC DATA

Hippolyte Verninas, Hadrien Hendrikx & Thomas Moreau

INRIA

{hippolyte.verninas,hadrien.hendrikx,thomas.moreau}@inria.fr

ABSTRACT

With the growing scale of scientific simulation and infrastructure, part of the burden has shifted from producing scientific data to analyzing and storing it. Compression is a key step to expanding storage capacity, but requires dedicated techniques to enforce user-specified pointwise error bounds that ensure the validity of downstream scientific workflows. To further push the limits of compression, novel learned codecs have been developed, in particular for images and videos, but these methods often optimize for aggregated distortion and fail to meet error bound standards. Moreover, these methods often target fixed field dimensionality, failing to account for many scientific data use cases, or at the expense of large adaptation or retraining costs. In this work, we introduce the Learned Axial Tensor Compression (LATEC) method, an error-bounded codec with a dimension-agnostic learned predictor based on a GNN architecture. LATEC is pretrained on synthetic 2D and 4D fields, and evaluated with and without domain fine-tuning on 13 scientific fields in 3D to 5D. Results show that LATEC is consistently competitive with SZ3, achieving a 10.8% average improvement and greater consistency across all domains than other compressors. These results demonstrate that a shared learned predictor can serve as a single error-bounded codec across tensor dimensionalities, competitive with specialized compressors, while retaining pointwise error guarantees.

1 INTRODUCTION

Scientific simulation is central to understanding systems that are difficult, costly, or impossible to observe experimentally. As simulations scale, the limiting factor shifts from producing data to storing and analyzing it, because compute throughput grows faster than the I/O bandwidth and storage budgets available to write results out. A single density field of a gyrokinetic plasma simulation on a 64^5 grid, three space and two velocity axes, already occupies 4 GiB, and a production run relies on hundredth of such 5D cells over many time steps and ensemble members. The shortfall is absorbed by discarding data: checkpoints are written at a coarser cadence, outputs are thinned, and ensembles are kept only in summary form, which limits post-hoc analysis and forces costly re-runs. Lossy compression is the standard way to keep more of this data, for archiving and transfer, for restart checkpoints, and to make repeated quantitative analyses practical (Cappello et al., 2019).

What makes scientific compression different is not the size of the data but how it is reused. A compressed field may become the input to a restarted computation, a threshold query, an extremum search, or a derived physical diagnostic (Cappello et al., 2019; Lee et al., 2022). None of these consumers inspect the field visually, and none tolerates an unknown worst case: a single corrupted value can move a detected extremum, flip a threshold test, or destabilize a restart, whatever the average error. Domain scientists therefore accept a compressed file as a substitute for the original only when the codec guarantees a pointwise bound $\|\mathbf{x} - \hat{\mathbf{x}}\|_\infty \leq \varepsilon$ matching their own tolerance: it certifies every reconstructed value, and holds regardless of which analysis is run downstream. Aggregate criteria cannot play this role, since an RMSE constraint at the same tolerance still admits an isolated error as large as $\sqrt{n}\varepsilon$ in a field of n values, which is why recent scientific benchmarks evaluate maximum-error compliance separately from average and structural distortion (Reichelt et al., 2026). This bound is the minimal contract every such workflow shares; preserving specific derived quantities or topological features adds application-specific constraints on top of it (Yan et al., 2023).

Established error-bounded scientific compressors provide strong baselines without relying on application-specific training. Table 1 summarizes the main trade-offs among the evaluated families. The SZ family predicts values from previously reconstructed neighbors or multilevel interpolation, then quantizes residuals so that prediction quality affects the rate but not the pointwise guarantee (Di & Cappello, 2016; Liang et al., 2023).

Later variants improve GPU execution and select or compose predictors and parameters automatically (Liu et al., 2024b; 2022; 2023b; 2024a). Other families make different structural choices: ZFP (Lindstrom, 2014) transforms independent blocks, MGARD (Ainsworth et al., 2019; Gong et al., 2023) uses a multilevel decomposition, TTHRESH (Ballester-Ripoll et al., 2020) exploits a higher-order tensor factorization, and SPERR (Li et al., 2023) combines wavelet coding with pointwise-error corrections. Their fixed analytic machinery makes these methods dependable and broadly applicable, but their coding efficiency depends on how well the chosen representation compacts each field.

Learned compression offers a way to exploit such structure, but existing architectures transfer poorly to scientific applications. Most progress has been driven by natural images and videos, where the reconstructed signal is ultimately judged through its rendering and codecs consequently optimize rate against an average or perceptual distortion. Their coding gains rest on nonlinear transforms paired with hierarchical or autoregressive entropy models (Ballé et al., 2017; 2018; Minnen et al., 2018), refined since for practical use (Tatwawadi et al., 2026) and, for video, through temporal prediction and learned motion representations (Jia et al., 2025b). Neither optimizes the criterion scientific workflows require, and both, like learned 3-D volume codecs, rely on convolutions whose kernels, feature layouts, and downsampling schedules assume a fixed number of dimensions. Learned scientific codecs similarly use blockwise or sparse autoencoders, constraint-aware reductions, 2-D super-resolution on slices, cross-field prediction, temporal graph models, or spatiotemporal latent models (Liu et al., 2021; Lee et al., 2022; Chung et al., 2025; Liu et al., 2023a; Jia et al., 2025a; Li et al., 2026; 2025a). Several add a residual correction stage to recover an error guarantee, but their learned component remains tied to a modality, dimensionality, or set of correlated variables. Many neural-field and autoencoder approaches are also fitted or trained anew for each dataset or domain (Hayne et al., 2021; Lu et al., 2021; Huang & Hoefler, 2023; Dupont et al., 2022). This cost is difficult to sustain when simulations expose many heterogeneous fields or when no representative training set is available.

A data-driven compressor for scientific tensors must therefore learn recurring structure without giving up the guarantees and broad applicability of predictive coding. Preserving native correlations requires exposing every tensor axis rather than flattening the data into slices, which privileges selected axes and discards correlations across the others. Yet naively extending a convolutional neighborhood of width k to D dimensions requires k^D entries and a dimension-specific architecture. The compressor must also generalize without retraining for each new field. Sharing learned operations across axes, as in dimension-free axial networks (Kim et al., 2025), provides a route to satisfying both requirements.

We introduce *Learned Axial TENSOR Compression* (LATEC), an error-bounded predictive codec for scientific tensors of arbitrary dimensionality. LATEC uses learned prediction to improve coding efficiency while retaining the deterministic pointwise guarantee of residual-based scientific compressors. A single model, pretrained exclusively on synthetic 2-D and 4-D fields, transfers to 3-D, 4-D, and 5-D scientific data with no target-domain fine-tuning. Across 13 scientific fields in three to five dimensions, LATEC is the most rate-efficient compressor with a pointwise guarantee on 9 fields. Relative to SZ3, it reduces the rate by 10.8% on average, while its largest compression overhead on any field is 18.9%.

Our contributions are:

- **A dimension-agnostic GNN predictor**, which follows a coarse-to-fine schedule the decoder reproduces exactly, and applies without dimension-specific redesign, slicing, or flattening.
- **Scale-guided gating**, which uses the GNN-predicted residual scale to select between the learned and classical predictions, allowing fine-grained control while only adding a few bytes of overhead.

Table 1: Compressor comparison (†: data-dependent).

Compressor	Speed	Compression quality	PWE bound	Dim. agnostic
SZ3	✓✓	✓	✓	✓
SPERR	✓	✓✓	✓	×
HPEZ	✓✓	✓	✓	×
ZFP	✓✓	×	✓	×
TTHRESH	✓	†	×	✓
LATEC (ours)	✓	✓✓	✓	✓

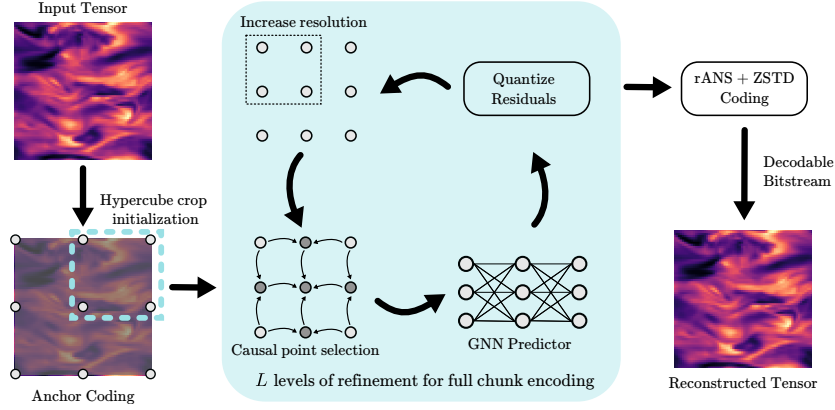


Figure 1: The end-to-end codec. After the anchors are encoded, each chunk covers one anchor cell and undergoes K levels of coarse-to-fine refinement. At every level, causally selected points are predicted by the gated GNN predictor, their residuals quantized under the prescribed error bound, and the resolution is increased to predict intermediate values. The resulting symbols are coded with rANS and framed with Zstandard.

- **A synthetic field generator for domain-agnostic pretraining**, which samples local correlation, multiple scales, anisotropy, and sharp interfaces without emulating any governing equation. It is the sole pretraining signal, and also a stronger initialization than random when target-domain data is available for fine-tuning.
- **An open, extensible benchmark for error-bounded tensor compression**, which compares LATEC with five established compressors under both matched pointwise error and PSNR.

2 ERROR-BOUNDED LEARNED COMPRESSION

LATEC separates the two roles of predictive compression: the coarse-to-fine residual coding inherited from the SZ family (Di & Cappello, 2016; Liang et al., 2023; Liu et al., 2024a) provides the error guarantee (Section 2.1), and learned prediction improves coding efficiency (Section 2.2). Figure 1 gives an overview of the resulting pipeline. Throughout, $\mathbf{x} \in \mathbb{R}^{n_1 \times \dots \times n_D}$ is a scalar field on the lattice $\Omega = \prod_{j=1}^D \{1, \dots, n_j\}$, $y_p \in [0, 1]$ is the value at $p \in \Omega$ after normalization to the field’s range, and the user-supplied range-relative pointwise-error bound becomes a tolerance $\varepsilon > 0$ in these coordinates.

2.1 ERROR-BOUNDED PREDICTIVE CODING

Predictive compression records, for each point of the field, not its value but how far that value departs from one that can already be computed from what has been decoded so far. The codec visits the points of Ω in a prescribed order: at point p it forms a prediction $\hat{\mu}_p$ from previously reconstructed values, codes the residual $y_p - \hat{\mu}_p$, and reconstructs $\hat{y}_p$, which then joins the context available to later points. Because predictions are computed from reconstructed values rather than original ones, the decoder reproduces the same sequence of states from the bitstream alone, which is what makes the loop closed. Compression comes from the residuals rather than the values: a predictor that fits the field concentrates them near zero, and the entropy coder then spends few bits per point. The tolerance ε enters only where the residuals are quantized, which is what makes the scheme error-bounded.

Anchors and the coarse-to-fine schedule. The visiting order must give every prediction a context the decoder already holds. LATEC adopts the dyadic, coarse-to-fine order of SZ-family multidimensional interpolation. The *anchor* lattice, the sublattice of stride $s_K = 2^K$ along every axis, is coded first and with no prediction at all: its values are quantized and coded directly. Anchors are the only values coded without context, and they bootstrap all the others. The K levels that follow, indexed by their stride $s_i = 2^i$ down to $s_0 = 1$, each halve the stride and predict the midpoints between values the previous level made available. Within a level, points whose neighbors lie along a single axis are coded before those requiring several axes, which splits each level into $2D - 1$ *sub-stages* (Appendix A.1),

giving a total of $(2D - 1)K$ stages. The anchor lattice also partitions the domain into *chunks*, cells of edge s_K that these levels are coded in one at a time (Section 2.4); one stage restricted to one chunk is a *chunk-stage*, the unit at which the gate of Section 2.2 is chosen. As errors at coarse levels propagate into every later prediction, these levels use a tighter tolerance than requested: level i codes at $\varepsilon_i = \varepsilon \rho^i$, with $0 < \rho \leq 1$, so the anchors at $i = K$ code at the tightest tolerance ε_K , and the finest level at the requested tolerance $\varepsilon_0 = \varepsilon$.

Quantization and the error guarantee. The prediction supplies a reconstruction center while residual quantization enforces the bound. The encoder converts the residual to the nearest quantization index,

$$q_p = \text{round} \left(\frac{y_p - \hat{\mu}_p}{2\varepsilon_i} \right), \quad (1)$$

and, for an in-range index, both sides reconstruct the value with exactly the arithmetic used for encoder-side verification:

$$\hat{y}_p = \hat{\mu}_p + 2\varepsilon_i q_p. \quad (2)$$

Rounding to the nearest level of a grid of spacing $2\varepsilon_i$ leaves an error of at most ε_i , and the encoder verifies $|y_p - \hat{y}_p| \leq \varepsilon_i$ explicitly for every quantized value. Note that **this guarantee is a property of the quantizer**, independent of $\hat{\mu}_p$: the quantization grid is centered on the prediction, so a poor estimate yields a large index and costs bits, but cannot increase the error. The only values left uncovered are those whose index exceeds the coded range or whose verification fails, and so are coded separately (Appendix B.2).

Entropy coding. The retained indices are coded with range asymmetric numeral systems (rANS) (Duda, 2013) and framed with Zstandard (Collet & Kucherawy, 2021). Alongside $\hat{\mu}_p$, the predictor returns a scale $\hat{\beta}_p$ parameterizing a Laplace distribution over the residual at p , which serves as the entropy model. Appendix B.4 details the setup, and Figure E.1 compares it with Huffman coding and a fixed scale.

2.2 AXIAL GNN PREDICTOR

Classical codecs use a fixed interpolation scheme, with coefficients chosen in advance and a shape tied to the number of axes. LATEC replaces the predictor with a GNN which builds an embedding for each reconstructed point and reuses these embeddings to predict the mean and scale for the points coded next. Because the schedule is unchanged, **encoder and decoder still see identical causal context** in any number of dimensions. Figure 2 shows one prediction step: line embeddings formed from the causal neighbors, pooling per axis and then across axes, a head that outputs (μ_p, β_p) , and a state update from the reconstructed value.

When coding a point $p \in \Omega$, the GNN may use only values reconstructed at earlier stages. It reaches them through a state $E \in \mathbb{R}^{|\Omega| \times D \times d}$, holding one embedding of width d per already-coded point and per lattice axis. The causal context of p is the nearest such point on either side of each of the D axis-aligned lattice lines through p . For each available line ℓ and axis j , the GNN combines the neighbor embeddings $E_{\ell,j}^\pm$ with context embeddings $Z_{\ell,j}^\pm$ that encode their direction and geometry. A line provides two-sided context when a reconstructed point is available on each side of p , and one-sided context when only a single point is available, as at a domain edge. Separate MLPs handle these two cases:

$$M_{\ell,j} = \begin{cases} \phi_{\text{bi-dir}}([E_{\ell,j}^-, Z_{\ell,j}^-, E_{\ell,j}^+, Z_{\ell,j}^+]), & \text{two-sided,} \\ \phi_{\text{dir}}([E_{\ell,j}^\sigma, Z_{\ell,j}^\sigma]), & \text{one-sided.} \end{cases} \quad (3)$$

Each available line ℓ produces one embedding $M_{\ell,j}$ for every axis j . Let $\mathcal{I}_p$ index the available lines through p , with $|\mathcal{I}_p| \leq D$. Following graph-attention pooling (Veličković et al., 2018), learned single-query maps $\psi_{\text{direction}}$ and ψ_{axis} first aggregate line embeddings separately for each axis, then aggregate the resulting per-axis contexts:

$$C_{p,j} = \psi_{\text{direction}}(\{M_{\ell,j}\}_{\ell \in \mathcal{I}_p}), \quad H_p = \psi_{\text{axis}}(\{C_{p,j}\}_{j=1}^D). \quad (4)$$

Pooling up to D line embeddings for each of D axis slots, then the D resulting axis contexts, costs $\mathcal{O}(D^2)$ per query at fixed embedding width. Because the line-embedding and pooling maps are shared across axes, the learned parameter count is independent of D , consistent with the dimension-free principle of axial neural networks (Kim et al., 2025).

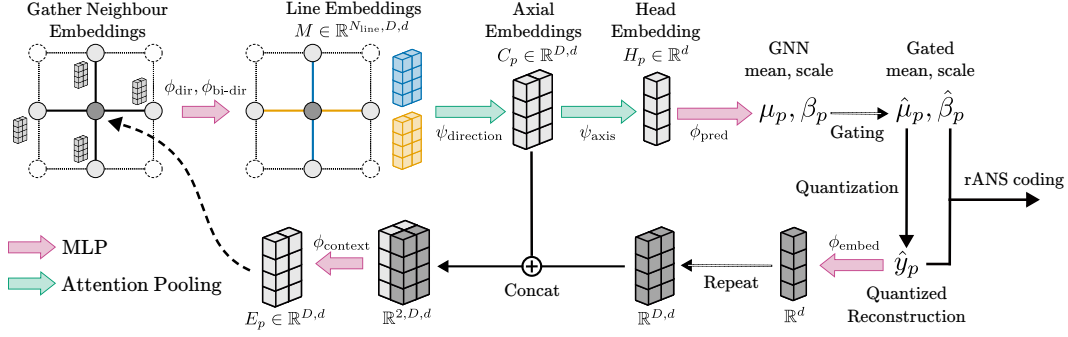


Figure 2: Causal axial GNN at one prediction stage. Line embeddings are pooled into one embedding per axis (two are shown for the 2-D case), then into the head that predicts (μ_p, β_p) , which gives $(\hat{\mu}_p, \hat{\beta}_p)$ after gating. The residual is quantized and entropy-coded, and the reconstructed value yields the embedding that updates the state E .

The prediction head also receives the stage tolerance ε_i . Conditioned on H_p and $\log_2 \varepsilon_i$, **the head returns a mean and Laplace scale**,

$$(\mu_p, \log_2 \beta_p) = \phi_{\text{pred}}([H_p, \log_2 \varepsilon_i]), \quad \mu_p \in [0, 1], \quad \beta_p \in [\varepsilon_i/16, 4096\varepsilon_i]. \quad (5)$$

Both the conditioning and the bounds are relative to ε_i , and the scale is predicted as $\log_2 \beta_p$: what matters for coding is the residual measured in quantization steps, not its absolute size. A given ratio β_p/ε_i then means the same thing at every level and for every requested tolerance, so a single trained head serves the whole schedule.

After quantization reconstructs $\hat{y}_p$, **encoder and decoder finalize the state at p by mixing that reconstructed value into its per-axis context**, $E_{p,j} \leftarrow \phi_{\text{context}}(C_{p,j}, \phi_{\text{embed}}(\hat{y}_p))$ for every axis j , with the learned maps detailed in Appendix B.1. Using $\hat{y}_p$ rather than y_p makes this closed-loop state reproducible at the decoder. Reusing these finalized embeddings at later stages propagates context through the hierarchy, even though each prediction uses a single graph-aggregation step.

2.3 SCALE-GUIDED GATING

The GNN predicts better than interpolation on average, but not everywhere: on locally smooth regions of some fields, a classical interpolant can be more accurate. LATEC therefore keeps three classical candidates $\{\mu_p^{(a)}\}_{a=1}^3$ alongside the GNN mean: cubic interpolation, and linear interpolation along the first or last axis on which p is a midpoint. Like the GNN, all three use only reconstructed values, so the decoder can evaluate any of them.

Which predictor wins changes across fields, stages, and regions, so a fixed choice is suboptimal, while a per-point selection flag would be too costly. One candidate a is fixed for a whole chunk-stage; what varies from point to point is only whether it is used in place of the GNN mean, **depending on the gating decision g_p made from the predicted scale β_p** . The decision function is parametrized for a given chunk-stage by a threshold $\varepsilon_i 2^T$, with $T \in \{2, \dots, 13\}$, on the same ε_i -relative grid as β_p , and a sign $d \in \{+1, -1\}$, which determines the direction of the threshold. The gate reads:

$$g_p = 1[d\beta_p < d\varepsilon_i 2^T] \quad \hat{\mu}_p = g_p \mu_p^{(a)} + (1 - g_p) \mu_p, \quad \hat{\beta}_p = 2^{-s g_p} \beta_p, \quad (6)$$

At the replaced points, the scale β_p was predicted for the GNN residual, so it would overestimate the residual given by the classical interpolant. The gate therefore multiplies it by a factor of 2^{-s} , $s \in \{0, 2, 4, 6\}$, so that the entropy model matches the smaller residual of the classical mean.

A gate is therefore described by four discrete choices, (a, d, T, s) . For each chunk-stage, the encoder estimates the code length of the stage’s residuals under the GNN alone and under each of the 288 combinations, and keeps the cheapest. The gate stays off unless some combination is strictly cheaper than the GNN alone (Appendix B.3). The selected parameters (a, d, T, s) are transmitted as one 16-bit word per chunk-stage. The decoder, which recomputes β_p , then rebuilds the mask g_p on its own: the choice of predictor is made **per point**, whereas only one word is transmitted **per chunk-stage**. Figures E.3 and E.5 compare this combined predictor with fixed interpolation and show the gate’s selections across fields.

2.4 CHUNKED EXECUTION FOR BOUNDED MEMORY

Learned prediction pays for its coding gain with per-point state that a classical codec does not carry. At the sizes where compression matters most, this state decides whether the codec can run at all. The state update finalizes one embedding of width d for every point and every axis, so retaining that field for a whole tensor would cost $\mathcal{O}(|\Omega|Dd)$: at the deployed width $d = 64$ in float32, $64D$ times the size of the input, or 1.25 TiB for the 4 GiB 64^5 field of the introduction. Coding one chunk at a time and discarding its embeddings on completion instead gives active storage $\mathcal{O}(VDd + |\Omega|)$, a neural term over the extended chunk volume V plus the scalar reconstruction buffer any codec must hold. The neural term VDd is then fixed by the configured chunk size and independent of the input, so **total memory grows with one scalar per point** rather than with Dd of them. Section 3 confirms this scaling empirically.

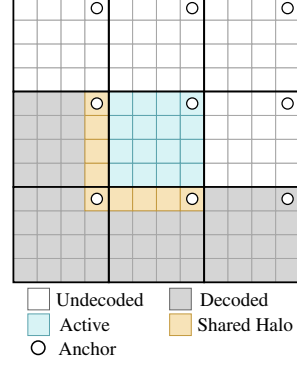


Figure 3: Chunking in 2-D.

Chunks are visited in raster order, and each inherits its low-coordinate faces from already reconstructed neighbors, as shown in Figure 3. This extended-chunk partition gives every point a single coding owner while supplying the boundary values later chunks need. Halo embeddings are rebuilt from within the current chunk rather than carried across the seam, so no learned state is retained between chunks (Appendix B.1).

2.5 DOMAIN-AGNOSTIC TRAINING ON SYNTHETIC FIELDS

Objective. The predictor is trained to minimize the bitrate it determines: the code length, in bits, that the entropy model parameterized by its predicted mean and scale assigns to each quantization index of Equation (1). Two substitutions make that cost cheap to evaluate and differentiable. First, the closed loop of deployment is replaced by teacher forcing: training follows the causal stage order but supplies later stages with $\text{clip}(y_p + \eta_p, 0, 1)$, where $\eta_p \sim \mathcal{U}[-\varepsilon, \varepsilon]$ has the same support as the reconstruction error it stands in for, so the predictor sees imperfect context without the encoder running. Second, the mass of the cell selected by round is replaced by the mass of the width- 2ε interval centered on the continuous residual, which takes the rounding out of the graph and leaves gradients on both the mean and the scale. Writing $F_{\text{Lap}}(\cdot; 0, \beta_p)$ for the zero-mean Laplace CDF with predicted scale β_p , the objective over the set $\mathcal{H}$ of non-anchor positions is

$$\mathcal{L} = \frac{1}{|\mathcal{H}|} \sum_{p \in \mathcal{H}} -\log_2[F_{\text{Lap}}(y_p - \mu_p + \varepsilon; 0, \beta_p) - F_{\text{Lap}}(y_p - \mu_p - \varepsilon; 0, \beta_p)]. \quad (7)$$

For $|y_p - \mu_p| \geq \varepsilon$ a logarithmic continuation of the Laplace tail limits extreme-residual gradients. The gate and the deployed entropy-table selection are decisions taken by the encoder at inference and are excluded from training. Appendix C gives the training details.

Synthetic generation. We train the predictor exclusively on fields synthesized on the training device, as done for tabular and time-series foundation models (Hollmann et al., 2025; Qu et al., 2025; Xie et al., 2026), which removes the need for a representative target-domain corpus, permits unlimited online resampling, and rules out target-domain leakage. The generator emulates no governing equation, and transfer is not expected to follow from any resemblance between its output and a simulated field. What the predictor sees at each query is a causal neighborhood a few points wide, and what sets the residual there is local structure: how fast the field decorrelates, whether it decorrelates at the same rate along every axis, how its values are distributed, and whether an interface crosses the neighborhood. The generator is built to span those properties rather than to imitate any physics.

Each realization draws three independent choices, one per property, listed in Table C.1. The spectrum is either smooth, white noise filtered by an anisotropic Gaussian spectrum at two correlation scales, or turbulent, a radial power law repeatedly warped by a smooth random displacement; between them they supply local correlation at several scales and anisotropy. A monotone value warp then varies the marginal distribution, and a number of random hyperplane jumps inserts sharp interfaces. The three choices give $2 \times 3 \times 4 = 24$ discrete construction types, with continuous parameters resampled within each.

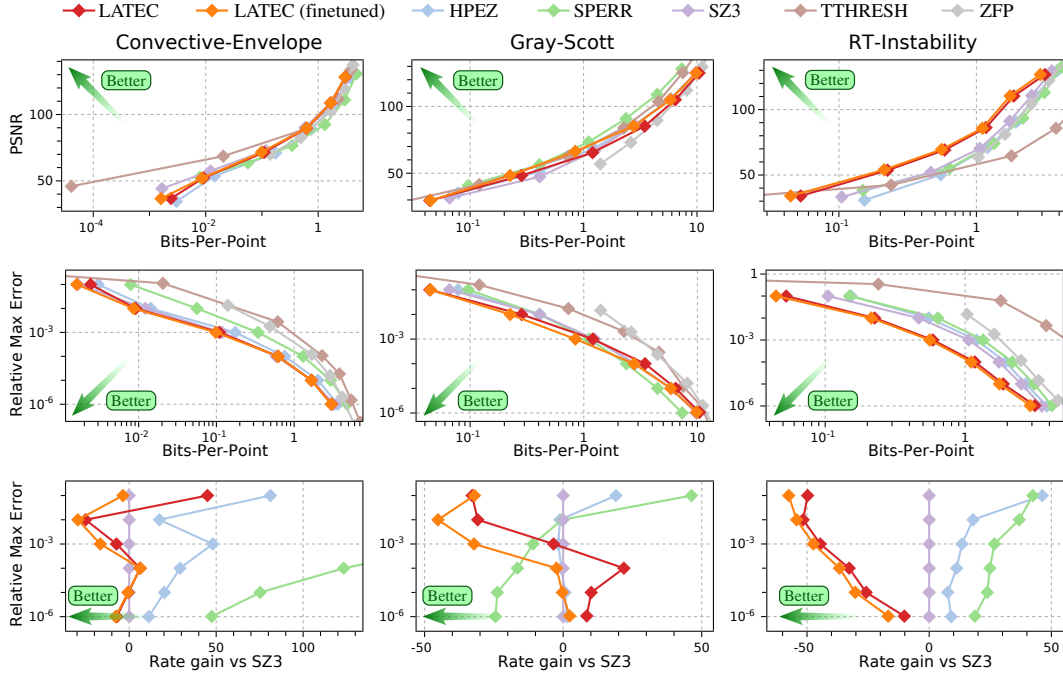


Figure 4: Rate-quality results on three datasets from The Well. We compare PSNR and RME against BPP and the relative BPP difference with SZ3.

A batch mixes 128×128 2-D and 16^4 4-D fields, resampled at every optimization step, in numbers that give the two dimensionalities equal weight in the loss. Half the 2-D steps split the field into two chunks with a randomized split axis and traversal order, so the predictor is also trained under the halo conditions of Section 2.4 rather than on whole fields alone. Appendix C gives the equations, parameter distributions, boundary conditions, and normalization.

3 CROSS-DIMENSIONAL EVALUATION

An open benchmark for error-bounded tensor compression. We release the evaluation as a Benchopt benchmark (Moreau et al., 2022), where datasets, compressors, and metrics are independent components, so a new field or codec plugs in without touching the rest.¹ It covers 13 scalar fields in three to five dimensions, drawn from SDRBench (Zhao et al., 2020), The Well (Ohana et al., 2024), the 5-D gyrokinetic simulations of GyroSwin (Paischer et al., 2025) and Gysela (Grandgirard et al., 2006), and 5-D synthetic data (Appendix D.3), listed with their shapes and sizes in Table D.1. The 5-D fields test whether a codec retains correlations across more than three axes without a privileged slicing axis. The predictor is pretrained on synthetic 2-D and 4-D fields only (Section 2.5), so 3-D and 5-D are unseen at training.

For each field and operating point we report compressed size in bits per point (BPP), the achieved range-relative maximum error (RME), and the peak signal-to-noise ratio (PSNR), defined in Appendix D.1. We compare LATEC against SZ3, SPERR, HPEZ, ZFP, and TTHRESH, each under its own error-control configuration. Because TTHRESH controls aggregate error rather than RME, its matched-RME values are estimates, included in the ranking but marked with an asterisk. We also evaluated the learned CAESAR compressor (Li et al., 2025a), but it performed poorly under our evaluation regime, so we report its results separately in Appendix D.4. Other learned methods, including AE-SZ (Liu et al., 2021) and SRN-SZ (Liu et al., 2023a), are excluded because their code is not publicly available. We evaluate the pretrained model on all 13 fields and, wherever a separate training split exists, namely each evaluated The Well dataset and GyroSwin, a checkpoint fine-tuned on that split alone and evaluated on a held-out test trajectory (Section 2.5 and Appendix C).

¹The benchmark code is included in the supplementary material.

Table 2: Average percentage BPP difference from SZ3 across matched relative maximum error (RME) targets for 3-D, 4-D, and 5-D datasets. Negative values are rate savings and lower is better. * marks estimated RME values because the compressor provides no bound, and “/” marks a dataset with no separate training split, hence no fine-tuned checkpoint. **Bold** is best, underlined second-best.

Method	Euler-Multi-Quadrants	Gray-Scott	Hurricane ISABEL	Miranda	S3D	Convective-Envelope	Gravity-Cooling	MHD-64	RT-Instability	Supernova-Explosion	GyroSwin	Gysela	Synthetic 5D
SZ3	<u>0</u>	0	<u>0</u>	0	0	<u>0</u>	0	0	0	0	<u>0</u>	0	<u>0</u>
ZFP	976.8	525.6	425.2	2148.0	4580.1	2366.1	4827.5	493.3	260.5	4369.6	182.2	2386.1	209.8
SPERR	26.0	<u>-5.0</u>	108.1	4.7	0.3	195.1	20.7	5.7	28.7	138.7	16.1	20.9	14.0
HPEZ	-4.4	2.9	24.6	<u>-4.8</u>	2.8	34.6	19.8	10.3	17.6	84.5	3.7	177.6	12.8
TTHRESH*	890.8	112.5	1293.2	89.6	-14.8	654.3	265.5	120.5	547.9	2912.9	26.8	-31.4	69.5
LATEC	18.9	-4.7	-14.8	-11.2	<u>-14.2</u>	2.0	<u>-3.0</u>	<u>-19.0</u>	<u>-35.9</u>	<u>-9.8</u>	1.9	<u>-3.1</u>	-47.3
LATEC (fine-tuned)	10.5	-18.5	/	/	/	-8.5	-21.8	-22.6	-40.6	-21.9	-2.0	/	/

Rate-error results. At matched RME, pretrained LATEC is the most rate-efficient compressor on 7 of the 13 fields, using fewer bits than each of SZ3, ZFP, SPERR, HPEZ and TTHRESH (Figure 4 and table 2). Among compressors with a pointwise guarantee, which excludes the estimated TTHRESH entries, it is the most rate-efficient on 9 fields. Pairwise, it beats ZFP on all 13 fields, SPERR and HPEZ on 12, the estimated TTHRESH entries on 11, and SZ3 on 10; three of its four losses to a guaranteed-bound baseline are within 2.0 percentage points, and only on Euler-Multi-Quadrants is the gap substantial. With the fine-tuned checkpoint on the eight adapted fields and the pretrained checkpoint elsewhere, LATEC is best on 10 of the 13 fields. On the 5-D fields, it is on par with SZ3 (-3.1% on Gysela, $+1.9\%$ on GyroSwin) while coding them in their native coordinates; SZ3 and TTHRESH also receive the native shape there, whereas ZFP, HPEZ, and SPERR must group axes (Appendix D.3).

Error-bound compliance. LATEC meets the requested pointwise bound on every field and at every target, up to floating-point round-off. The achieved RME never exceeds the target by more than 1.3×10^{-7} of the field range, which corresponds to the float32 resolution of the normalized reconstruction (Appendix B.2) and is comparable to the excess measured for SPERR.

Consistency across fields. LATEC’s aggregate advantage is not confined to a single error regime. It achieves the best dataset-averaged rate from RME 10^{-1} through 10^{-5} . However its lead narrows as the bound tightens and SZ3 has a small advantage at 10^{-6} (Figure 5). HPEZ and SPERR, in contrast, incur an average overhead at every target, and their gap from SZ3 widens markedly as the bound loosens. Averaged over the fields of Table 2, LATEC is the only evaluated compressor whose mean rate falls below SZ3’s, with a 10.8% mean and 9.8% median saving, and its worst field costs 18.9% more than SZ3. The other compressors have much larger worst cases, from HPEZ’s 177.6% on Gysela to 2912.9% for the estimated TTHRESH entries on supernova explosion, and ZFP averages 1827.0%.

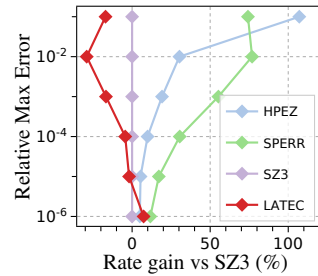


Figure 5: Dataset-averaged rate difference from SZ3 by RME target.

Rate-distortion results. The PSNR curves in Figure 4 provide a secondary view of average reconstruction error at a given rate. At matched PSNR, pretrained LATEC uses fewer bits than SZ3 on 7 of the 13 fields and, with the fine-tuned checkpoint, is the best method on 7 of them; SPERR, HPEZ, and especially TTHRESH remain stronger on several datasets (Table E.1 in Appendix E.1). The two views differ for a reason shared by the whole prediction-and-quantization family: uniform quantization at spacing $2\epsilon_i$ spreads the error budget evenly over every sample, which makes the L^∞ contract inexpensive to satisfy but is not how bits would be allocated to minimize squared error, so transform methods truncating against an L^2 criterion are favored by PSNR. The matched-RME

Table 3: Encoding and decoding throughput and peak host memory across dimensionalities, at a relative error bound of 10^{-4} . Parenthesized values are fitted intercepts (overheads), in seconds for throughput and MiB for RAM. The encoding-time, decoding-time, host-RAM, and GPU-VRAM regressions for 4-D inputs are in Figure E.7.

Method	Encoding throughput MiB/s (+s)		Decoding throughput MiB/s (+s)		Peak host RAM scaling MiB/MiB (+MiB)	
	3-D	5-D	3-D	5-D	3-D	5-D
LATEC	9.7 (+4)	4.5 (+13)	10.7 (+0)	5.2 (+0)	1.77 (+3441)	1.88 (+4091)
SZ3	114 (+0)	108 (+0)	138 (+0)	142 (+0)	3.31 (+100)	3.36 (+52)
SPERR	16 (-3)	19 (+7)	24 (-2)	33 (+4)	11.69 (-203)	9.77 (+516)
TTHRESH	9 (+5)	10 (+1)	15 (-1)	16 (+0)	10.98 (+43)	11.04 (+7)

comparison in Table 2 remains the primary evaluation of compression rate under the requested pointwise bound.

Effect of the synthetic pretraining. Fine-tuning lowers LATEC’s BPP on all eight adapted datasets, at matched RME and at matched PSNR (Tables 2 and E.1). These gains come from the pretrained initialization rather than from target data alone: training the same architecture from random initialization on each of the seven evaluated datasets from The Well gives a substantially higher average bitrate early in training, and the gap persists at the final checkpoint (Figure 6).

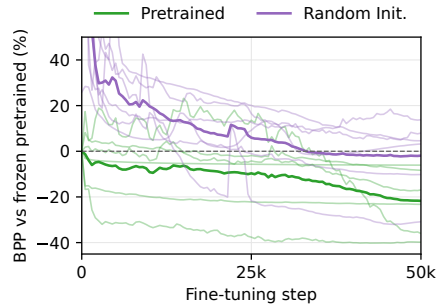


Figure 6: BPP of the held-out test tensor during fine-tuning on The Well, relative to the frozen pretrained model, for synthetic pretraining versus random initialization.

Memory usage. Chunked execution keeps memory growth close to one scalar per input value. Peak host RAM usage grows by 1.77-1.88 MiB and peak GPU VRAM by 1.02-1.07 MiB per MiB of input from 3-D to 5-D inputs, on top of a fixed overhead, with host-memory fits summarized in Table 3 and 4-D host and GPU measurements shown in Figure E.7. Without chunking, the slope would be $64D$, that is 192 to 320. That overhead is amortized at the sizes where compression matters: at a 4 GiB input, peak host RSS is 10.5 GiB for LATEC against 43.5 GiB for TTHRESH.

Throughput. The learned predictor is the main cost here. LATEC encodes at 4.5-9.7 MiB/s and decodes at 5.2-10.7 MiB/s, within a factor of about three of TTHRESH but 12-24 times slower than SZ3 for encoding and 13-27 times slower for decoding. Throughput optimization was not a primary focus of this work, and GPU utilization during compression remains around 50%. This leaves room for more efficient kernel-level implementations, better cache utilization, and greater exploitation of the per-stage parallelism exposed by the causal schedule.

4 CONCLUSION

This work prioritizes compression rate under a strict error bound over throughput, so LATEC is slower than classical compressors and relies on a GPU for practical decoding speed. However this gap could be tightened with cache optimizations, custom kernels and other code optimizations. Because decoding must reproduce the encoder’s predictions bitwisely, streams are also tied to the platform that produced them, a restriction deterministic integer inference would lift. Finally, LATEC’s advantage is smaller at matched PSNR, where transform-based compressors remain strong; its gate could naturally be extended to include transform-based predictions, a promising direction for future work.

LATEC shows that one learned predictor, pretrained only on synthetic fields, can code scientific tensors of three to five dimensions in their native shapes at a lower rate than established error-bounded codecs, while strictly enforcing the pointwise bound these workflows require. It therefore removes the two costs that have kept learned compression out of these workflows: collecting a representative training corpus for each domain, and giving up the pointwise error guarantee. We hope this encourages general-purpose learned compressors for scientific data.

AI USE

Large language models were used as assistants for writing code, surveying the literature, drafting text, and proofreading. All research ideas, the method, and the experimental design originate from the authors, and all figures were produced by the authors without generative AI. The authors reviewed and verified all AI-assisted content and take full responsibility for the paper.

REPRODUCIBILITY

The complete source code of LATEC, including the synthetic training stream, training scripts, and the BenchOpt benchmark, is provided in the supplementary material and will be released publicly, together with the pretrained and fine-tuned model checkpoints. Architecture and training details are given in Section 2.5 and appendix C, the evaluation metrics and rate-difference computation in Appendices D.1 and D.5, and the benchmark fields in Table D.1. All evaluation data are publicly available (SDRBench, The Well, and GyroSwin), except the Gysela field; the synthetic 5-D field is generated by the released code (Appendix D.3).

ACKNOWLEDGMENTS

This work was supported from a national grant attributed to the ExaDoST project of the NumPEX PEPR program, under the reference ANR-22-EXNU-0004.

REFERENCES

- Mark Ainsworth, Ozan Tugluk, Ben Whitney, and Scott Klasky. Multilevel techniques for compression and reduction of scientific data—the multivariate case. *SIAM Journal on Scientific Computing*, 2019.
- Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. In *International Conference on Learning Representations*, 2017.
- Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, and Nick Johnston. Variational image compression with a scale hyperprior. In *International Conference on Learning Representations*, 2018.
- Rafael Ballester-Ripoll, Peter Lindstrom, and Renato Pajarola. TTHRESH: Tensor compression for multidimensional visual data. *IEEE Transactions on Visualization and Computer Graphics*, 2020.
- Franck Cappello, Sheng Di, Sihuan Li, Xin Liang, Ali Murat Gok, Dingwen Tao, Chun Hong Yoon, Xin-Chuan Wu, Yuri Alexeev, and Frederic T. Chong. Use cases of lossy compression for floating-point data in scientific data sets. *The International Journal of High Performance Computing Applications*, 2019.
- Matthias Chung, Richard Archibald, Paul Atzberger, and Jack Michael Solomon. Sparse L^1 -autoencoders for scientific data compression. *Journal of Machine Learning for Modeling and Computing*, 2025.
- Yann Collet and Murray Kucherawy. Zstandard compression and the ‘application/zstd’ media type. RFC 8878, 2021.
- Sheng Di and Franck Cappello. Fast error-bounded lossy HPC data compression with SZ. In *IEEE International Parallel and Distributed Processing Symposium*, 2016.
- Jarek Duda. Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding. *arXiv*, 2013.
- Emilien Dupont, Hrushikesh Loya, Milad Alizadeh, Adam Goliński, Yee Whye Teh, and Arnaud Doucet. COIN++: Neural compression across modalities. *Transactions on Machine Learning Research*, 2022.

- Qian Gong, Jieyang Chen, Ben Whitney, Xin Liang, Viktor Reshniak, Tania Banerjee, Jaemoon Lee, Anand Rangarajan, Lipeng Wan, Nicolas Vidal, Qing Liu, Ana Gainaru, Norbert Podhorszki, Richard Archibald, Sanjay Ranka, and Scott Klasky. MGARD: A multigrid framework for high-performance, error-controlled data compression and refactoring. *SoftwareX*, 2023.
- V. Grandgirard, Y. Sarazin, X. Garbet, G. Dif-Pradalier, Ph. Ghendrih, N. Crouseilles, G. Latu, E. Sonnendrücker, N. Besse, and P. Bertrand. GYSELA, a full-f global gyrokinetic semi-Lagrangian code for ITG turbulence simulations. *AIP Conference Proceedings*, 2006.
- Lucas Hayne, John Clyne, and Shaomeng Li. Using neural networks for two dimensional scientific data compression. In *IEEE International Conference on Big Data*, 2021.
- Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeyer, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 2025.
- Langwen Huang and Torsten Hoefler. Compressing multidimensional weather and climate data into neural networks. In *International Conference on Learning Representations*, 2023.
- Wenqi Jia, Zhewen Hu, Youyuan Liu, Boyuan Zhang, Jinzhen Wang, Jinyang Liu, Wei Niu, Stavros Kalafatis, Junzhou Huang, Sian Jin, Daoce Wang, Jiannan Tian, and Miao Yin. NeurLZ: An online neural learning-based method to enhance scientific lossy compression. In *ACM International Conference on Supercomputing*, 2025a.
- Zhaoyang Jia, Bin Li, Jiahao Li, Wenxuan Xie, Linfeng Qi, Houqiang Li, and Yan Lu. Towards practical real-time neural video compression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025b.
- Hyunsu Kim, Jonggeon Park, Joan Bruna, Hongseok Yang, and Juho Lee. Axial neural networks for dimension-free foundation models. In *Advances in Neural Information Processing Systems*, 2025.
- Jaemoon Lee, Qian Gong, Jong Choi, Tania Banerjee, Scott Klasky, Sanjay Ranka, and Anand Rangarajan. Error-bounded learned scientific data compression with preservation of derived quantities. *Applied Sciences*, 2022.
- Guozhong Li, Muhannad Alhumaidi, Spiros Skiadopoulos, Ibrahim Hoteit, and Panos Kalnis. Extreme error-bounded compression of scientific data via temporal graph autoencoders. *IEEE Transactions on Knowledge and Data Engineering*, 2026.
- Shaomeng Li, Peter Lindstrom, and John Clyne. Lossy scientific data compression with SPERR. In *IEEE International Parallel and Distributed Processing Symposium*, 2023.
- Xiao Li, Jaemoon Lee, Anand Rangarajan, and Sanjay Ranka. Foundation model for lossy compression of spatiotemporal scientific data. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2025a.
- Xiao Li, Liangji Zhu, Jaemoon Lee, Rahul Sengupta, Scott Klasky, Sanjay Ranka, and Anand Rangarajan. CAESAR: A unified framework for foundation and generative models for efficient compression of scientific data. *Applied Sciences*, 2025b.
- Xin Liang, Kai Zhao, Sheng Di, Sihuan Li, Robert Underwood, Ali M. Gok, Jiannan Tian, Junjing Deng, Jon C. Calhoun, Dingwen Tao, Zizhong Chen, and Franck Cappello. SZ3: A modular framework for composing prediction-based error-bounded lossy compressors. *IEEE Transactions on Big Data*, 2023.
- Peter Lindstrom. Fixed-rate compressed floating-point arrays. *IEEE Transactions on Visualization and Computer Graphics*, 2014.
- Jinyang Liu, Sheng Di, Kai Zhao, Sian Jin, Dingwen Tao, Xin Liang, Zizhong Chen, and Franck Cappello. Exploring autoencoder-based error-bounded compression for scientific data. In *IEEE International Conference on Cluster Computing*, 2021.

- Jinyang Liu, Sheng Di, Kai Zhao, Xin Liang, Zizhong Chen, and Franck Cappello. Dynamic quality metric oriented error bounded lossy compression for scientific datasets. In *International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.
- Jinyang Liu, Sheng Di, Sian Jin, Kai Zhao, Xin Liang, Zizhong Chen, and Franck Cappello. Scientific error-bounded lossy compression with super-resolution neural networks. In *IEEE International Conference on Big Data*, 2023a.
- Jinyang Liu, Sheng Di, Kai Zhao, Xin Liang, Zizhong Chen, and Franck Cappello. FAZ: A flexible auto-tuned modular error-bounded compression framework for scientific data. In *ACM International Conference on Supercomputing*, 2023b.
- Jinyang Liu, Sheng Di, Kai Zhao, Xin Liang, Sian Jin, Zizhe Jian, Jiajun Huang, Shixun Wu, Zizhong Chen, and Franck Cappello. High-performance effective scientific error-bounded lossy compression with auto-tuned multi-component interpolation. *Proceedings of the ACM on Management of Data*, 2024a.
- Jinyang Liu, Jiannan Tian, Shixun Wu, Sheng Di, Boyuan Zhang, Robert Underwood, Yafan Huang, Jiajun Huang, Kai Zhao, Guanpeng Li, Dingwen Tao, Zizhong Chen, and Franck Cappello. cuSZ-i: High-ratio scientific lossy compression on GPUs with optimized multi-level interpolation. In *International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024b.
- Yuzhe Lu, Kairong Jiang, Joshua A. Levine, and Matthew Berger. Compressive neural representations of volumetric scalar fields. *Computer Graphics Forum*, 2021.
- David Minnen, Johannes Ballé, and George D. Toderici. Joint autoregressive and hierarchical priors for learned image compression. In *Advances in Neural Information Processing Systems*, 2018.
- Thomas Moreau, Mathurin Massias, Alexandre Gramfort, Pierre Ablin, Pierre-Antoine Bannier, Benjamin Charlier, Mathieu Dagréou, Tom Dupré la Tour, Ghislain Durif, Cassio F. Dantas, Quentin Klopfenstein, Johan Larsson, En Lai, Tanguy Lefort, Benoît Malézieux, Badr Moufad, Binh T. Nguyen, Alain Rakotomamonjy, Zaccharie Ramzi, Joseph Salmon, and Samuel Vaiter. Benchopt: Reproducible, efficient and collaborative optimization benchmarks. In *Advances in Neural Information Processing Systems*, 2022.
- Ruben Ohana, Michael McCabe, Lucas Meyer, Rudy Morel, Fruzsina J. Agocs, Miguel Beneitez, Marsha Berger, Blakesley Burkhart, Stuart B. Dalziel, Drummond B. Fielding, Daniel Fortunato, Jared A. Goldberg, Keiya Hirashima, Yan-Fei Jiang, Rich R. Kerswell, Suryanarayana Maddu, Jonah Miller, Payel Mukhopadhyay, Stefan S. Nixon, Jeff Shen, Romain Watteaux, Bruno Régaldou-Saint Blancard, François Rozet, Liam Parker, Miles Cranmer, and Shirley Ho. The Well: a large-scale collection of diverse physics simulations for machine learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.
- Fabian Paischer, Gianluca Galletti, William Hornsby, Paul Setinek, Lorenzo Zanisi, Naomi Carey, Stanislas Pamela, and Johannes Brandstetter. GyroSwin: 5D surrogates for gyrokinetic plasma turbulence simulations. In *Advances in Neural Information Processing Systems*, 2025.
- Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. TabICL: A tabular foundation model for in-context learning on large data. In *International Conference on Machine Learning*, 2025.
- Tim Reichelt, Juniper Tyree, Milan Klöwer, Peter Dueben, Bryan N Lawrence, Allison H Baker, Sara Faghieh-Naini, Torsten Hoeffler, and Philip Stier. ClimateBenchPress (v1.0): a benchmark for lossy compression of climate data. *Geoscientific Model Development*, 2026.
- Kedar Tatwawadi, Parisa Rahimzadeh, Zhanghao Sun, Zhiqi Chen, Ziyun Yang, Sanjay Nair, Divija Hasteer, and Oren Rippel. What matters in practical learned image compression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

- Shifeng Xie, Vasilii Feofanov, Jianfeng Zhang, Themis Palpanas, and Ievgen Redko. CauKer: Classification time series foundation models can be pretrained on synthetic data. In *International Conference on Learning Representations*, 2026.
- Lin Yan, Xin Liang, Hanqi Guo, and Bei Wang. TopoSZ: Preserving topology in error-bounded lossy compression. *IEEE Transactions on Visualization and Computer Graphics*, 2023.
- Kai Zhao, Sheng Di, Xin Liang, Sihuan Li, Dingwen Tao, Julie Bessac, Zizhong Chen, and Franck Cappello. SDRBench: Scientific data reduction benchmark for lossy compressors. In *IEEE International Conference on Big Data*, 2020.

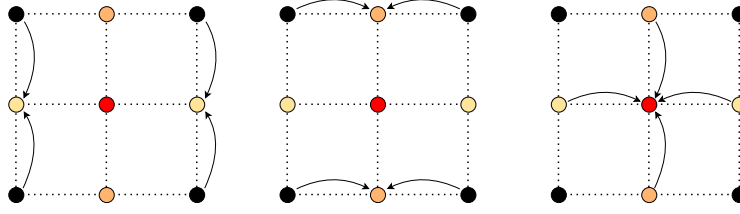


Figure A.1: The three sub-stages of one 2-D dyadic refinement. In 3-D there are five sub-stages: three for 1-D interpolation, one for 2-D, and one for 3-D.

A CAUSAL SCHEDULE AND PREDICTOR DESIGN

A.1 CAUSAL MULTIDIMENSIONAL SCHEDULE

The schedule has K dyadic levels and stage strides $s_i = 2^i$. Stage K codes the anchor lattice, of stride $s_K = 2^K$, directly against a zero predictor. Each refinement level $i = K - 1, \dots, 0$ halves the stride. For each nonempty midpoint-axis set $S \subseteq \{1, \dots, D\}$, a sub-stage comprises points whose offset $p_j - 1$ from the first lattice index is an odd multiple of s_i on axes in S and a multiple of $2s_i$ elsewhere:

$$\mathcal{M}_{i,S} = \{p : p_j - 1 \equiv s_i \pmod{2s_i} \ (j \in S), \ p_j - 1 \equiv 0 \pmod{2s_i} \ (j \notin S)\} \setminus \mathcal{K}_{>i,S}. \quad (8)$$

Here $\mathcal{K}_{>i,S}$ contains locations already decoded at coarser levels. The masks $\mathcal{M}_{i,S}$, one per nonempty S at each level, form a disjoint partition into $1 + K(2^D - 1)$ sets. Sub-stages are processed by increasing $|S|$, and only the D singleton sets ($|S| = 1$) are kept separate by axis; sets with $|S| > 1$ are merged by cardinality into a single sub-stage each, since same-cardinality sets never depend on one another. This groups the $2^D - 1$ masks at each level into $2D - 1$ sub-stages, for $1 + K(2D - 1)$ overall. Before a midpoint is predicted, its $\pm s_i$ neighbors along every axis in S have already been reconstructed. Figure A.1 illustrates the resulting anchor-edge-center order in 2-D; the same rule generalizes to any number of dimensions.

This construction instantiates in any number of dimensions the dependency ordering used by multi-dimensional interpolation schemes such as HPEZ, which first interpolate points with 1-D context and then those with multidimensional context (Liu et al., 2024a). Simultaneous midpoints on every nonempty subset of axes become explicit sub-stages, providing decoder-reproducible causal geometry to the learned predictor.

The per-stage tolerance $\varepsilon_i = \varepsilon \rho^i$ is parameterized by its coarsest factor r through $\rho = r^{1/(K-1)}$ for $K > 1$, so the coarsest refinement stage ($i = K - 1$) uses $r\varepsilon$ regardless of the number of refinement levels. This allocation affects rate, not the pointwise guarantee.

A.2 GEOMETRIC CONTEXT CONSTRUCTION

Index a candidate line by ℓ , and choose an oriented representative u_ℓ for its direction. If they exist, its nearest already reconstructed neighbors of p in the $+u_\ell$ and $-u_\ell$ directions are $r_{p,\ell}^+$ and $r_{p,\ell}^-$, respectively. For axis j , the neighbor embeddings used in Equation (3) are $E_{\ell,j}^\pm = E_{r_{p,\ell,j}^\pm}$, and their lattice distances are $\Delta_{p,\ell}^\pm = \|r_{p,\ell}^\pm - p\|_\infty \in \mathbb{N}$. Let $z_{\ell,j}^\pm$ be a learned lookup of the signed cosine $\pm u_{\ell,j} / \|u_\ell\|_2 \in \{0, \pm 1\}$. The context embeddings in Equation (3) collect the direction and distance features as $Z_{\ell,j}^\pm = [z_{\ell,j}^\pm, \log_2 \Delta_{p,\ell}^\pm]$ in the two-sided case. For a one-sided neighborhood, $\sigma \in \{+1, -1\}$ denotes the available side and $Z_{\ell,j}^\sigma = [z_{\ell,j}^\sigma, \sigma, \log_2 \Delta_{p,\ell}^\sigma]$. Missing neighbors and out-of-domain lines are masked before pooling.

A.3 AGGREGATION-LEVEL TRADEOFF

The axial GNN uses only axis-aligned lattice lines, corresponding to aggregation level $L_{\text{agg}} = 1$. We also considered $L_{\text{agg}} = 2$, which augments the D axial lines with $D(D - 1)$ two-axis diagonal lines. Each line contributes one embedding to every axis slot. At fixed embedding width, the D axial lines

therefore require $\mathcal{O}(D^2)$ embedding computations per query, while adding the diagonal lines raises this cost to $\mathcal{O}(D^3)$. Because the additional diagonal context yielded only marginal performance gains while making inference slower, all reported experiments use $L_{\text{agg}} = 1$.

B DEPLOYED CODEC AND IMPLEMENTATION DETAILS

B.1 CHUNK STATE AND DEPLOYMENT SETTINGS

The encoder and decoder use the same state lifecycle. First, they reconstruct the global anchor lattice. For each chunk in raster order, they form its extended chunk and exclude its inherited low-coordinate faces from coding ownership. These one-point-wide faces contain adjacent values already reconstructed by earlier chunks. The codec reads their values from the global reconstruction buffer and includes the corresponding points in the new chunk’s local stage sequence. When such a point r is finalized, its embedding is rebuilt as $E_{r,j} = \phi_{\text{mix}}(C_{r,j}, \phi_{\text{val}}(\hat{y}_r))$, where $C_{r,j}$ contains only context rebuilt during the current chunk. No finalized embedding from the preceding chunk is retained.

Within the chunk, each stage finalizes the preceding stage’s reconstructed points, computes the next predictions, and reconstructs the owned points from their residual symbols or escapes. Reconstructions are written to the global scalar buffer, including the chunk’s owned high faces. After the last stage, the chunk is marked complete and its embedding field and cached stage context are released. A subsequent chunk reads its inherited faces from that buffer and rebuilds their embeddings during its own stage sequence. This state reset limits the learned context carried across a seam, but keeps both sides synchronized without storing a global neural field.

Each chunk is one anchor cell, with edge equal to the anchor stride $s_K = 2^K$ along every axis, clipped at the domain boundary. The one-point shared faces add to this nominal volume. For the aggregation-level-one model, automatic refinement depths are 7, 5, and 4 in 3-D, 4-D, and 5-D, respectively, giving nominal chunk volumes of $128^3 = 2^{21}$, $32^4 = 2^{20}$, and $16^5 = 2^{20}$ points. These depths are chosen large enough to give each chunk ample coarse context, while keeping the per-chunk memory footprint moderate. The depth is capped by $\max(1, \lfloor \log_2 n_{\text{max}} \rfloor - 1)$ on small tensors, where $n_{\text{max}} = \max_j n_j$. The default coarse error factor is $r = 0.9$.

B.2 BOUNDED RECONSTRUCTION AND ESCAPES

For a finite radius R , an in-range quantization index from Equation (1) ($|q_p| < R$) is sent as $c_p = q_p + R \in \{1, \dots, 2R - 1\}$; $c_p = 0$ is an escape symbol. The encoder emits this escape symbol, followed by the original normalized float32 value in scan order, if the index is out of range or if verification of Equation (2) fails. Thus every non-escape value satisfies $|y_p - \hat{y}_p| \leq \varepsilon_i$ by explicit verification, and every escape value is reproduced exactly in normalized float32. Normalization and the stored reconstructed state use float32. Residual-index calculation and reconstruction accumulation use float64 before the candidate is cast to float32 and verified. Decoder reconstruction uses the same operations. Verification is performed in normalized coordinates. In exact arithmetic, inverse normalization multiplies this error by the original data range, so the normalized bound is the range-relative bound in original units. Normalization and inverse normalization introduce floating-point rounding, and the benchmark evaluates the final output error in float64.

This is the same separation of concerns used by predictive error-bounded compressors: prediction improves rate, whereas quantization and an escape path establish the bound (Di & Cappello, 2016; Liang et al., 2023). It also appears in learned scientific codecs: AE-SZ combines a blockwise autoencoder with SZ-style error-controlled coding (Liu et al., 2021), Lee et al. (2022) apply an error-bounded residual post-process after learned reduction, and SRN-SZ alternates neural super-resolution with quantized correction offsets (Liu et al., 2023a). In each case, the learned model improves the reconstruction estimate, but the explicit residual or correction representation supplies the pointwise guarantee.

B.3 GATE SELECTION AND TRANSMISSION

This section details the gate of Equation (6): its classical interpolators, the encoder search that selects one gate per chunk-stage, and how the selection is passed on to the decoder.

Classical interpolators. At stage i , a point p is the midpoint of its neighbors at distance s_i along each of its midpoint axes, the axes in its set S of Equation (8). Along one such axis, linear interpolation averages the two neighbors at $\pm s_i$, and cubic interpolation uses the four-tap stencil $(-1, 9, 9, -1)/16$ on the neighbors at $\pm s_i, \pm 3s_i$. Near a line end, where the $\pm 3s_i$ neighbor is missing, the cubic stencil falls back to a quadratic one, and beyond the last available neighbor both interpolators extrapolate. The three interpolators a are

- *cubic*: the cubic interpolations averaged over all midpoint axes in 3-D and below, and the cubic interpolation along the first midpoint axis in 4-D and above, where averaging many estimates was found to be less accurate.
- *first-axis linear*: linear interpolation along the first midpoint axis.
- *last-axis linear*: linear interpolation along the last midpoint axis.

They read only reconstructed values, so encoder and decoder compute identical candidates.

Gated mean and scale. The gate of a chunk-stage is a tuple (a, d, T, s) that consists of a classical interpolator a , a side $d \in \{\text{low}, \text{high}\}$, a threshold exponent $T \in \{2, \dots, 13\}$, and a scale shift $s \in \{0, 2, 4, 6\}$. It selects the points $g_p = 1$ of Equation (6): those with $\beta_p < \varepsilon_i 2^T$ for the low side, and the others for the high side. At these points, the gated mean $\hat{\mu}_p$ is the classical interpolation and the gated scale is $\hat{\beta}_p = 2^{-s} \beta_p$. Elsewhere, both the GNN mean and scale are kept. Since $\beta_p \in [\varepsilon_i/16, 4096\varepsilon_i]$, the largest exponent $T = 13$ lies above every scale, so its low side replaces the GNN mean on the whole chunk-stage, whereas smaller exponents replace it only on part of the chunk-stage. The shift is shared by all selected points rather than chosen per point, so $\hat{\beta}_p$ keeps the pointwise variation of β_p . Both $\hat{\mu}_p$ and $\hat{\beta}_p$ are then used exactly as the ungated outputs: the residual $y_p - \hat{\mu}_p$ is quantized under ε_i , and $\hat{\beta}_p$ selects its probability table (Appendix B.4).

Encoder search. For every nonempty chunk-stage, the encoder evaluates all $3 \times 2 \times 12 \times 4 = 288$ tuples and the GNN alone, and keeps the one with the smallest estimated code length. The estimated length of a stage is the sum over its points of the cost of the residual index under the Laplace model of Equation (9), after snapping $\hat{\beta}_p$ to the 64-level scale grid used for coding. A regular index costs its negative log probability, capped at 24 bits, and an out-of-range index costs 56 bits, a 24-bit escape marker plus its raw float32 value. The GNN alone is the baseline, and the gate is disabled unless some tuple is strictly cheaper. The search is exhaustive yet inexpensive: the points are binned by the interval between consecutive thresholds $\varepsilon_i 2^T$ containing their scale, and cumulative sums of per-bin costs give the cost of every threshold and side at once. The search compares residuals to the source values, which only the encoder holds, but it selects among predictors and therefore affects rate only. Quantization and verification enforce the error bound for any choice.

Transmission to the decoder. The selected tuple is packed into one 16-bit word: two bits for the interpolator, with zero meaning that the gate is off, one bit for the side, four bits for T , and four bits for s . The words of all chunk-stages are compressed with Zstandard at level 19 and stored base64-encoded in the stream header. At each chunk-stage, the decoder reads the word, computes the selected interpolation from reconstructed values, and applies Equation (6) to the scale β_p it recomputes from the GNN. It thus rebuilds the same mask, gated mean, and gated scale as the encoder without repeating the search, and no per-point flag is transmitted. Predictor-selection percentages in Figure E.5 count the points selected by these chunk-stage rules.

B.4 ENTROPY MODEL AND RANS CODING

The error-control procedure determines the symbols; the entropy model affects only their code length. For each point, the gate supplies the adjusted Laplace scale $\hat{\beta}_p$. We round it to one of 64 logarithmically spaced values in $[\varepsilon_i/16, 4096\varepsilon_i]$, denoted $\tilde{\beta}_p$, allowing encoder and decoder to select the same probability table without transmitting a floating-point scale per value. The probability

assigned to the point's index q_p is the mass of its quantization bin,

$$P(q_p | \tilde{\beta}_p) = F_{\text{Lap}}(2\varepsilon_i q_p + \varepsilon_i; 0, \tilde{\beta}_p) - F_{\text{Lap}}(2\varepsilon_i q_p - \varepsilon_i; 0, \tilde{\beta}_p). \quad (9)$$

At each stage, the encoder records the smallest power-of-two alphabet that covers the observed in-range indices and a selected core-tail Laplace-mixture model. From these two stage-level parameters and the rounded per-point scale, the decoder regenerates the same normalized integer CDF. rANS then encodes the residual or escape symbols using that CDF (Duda, 2013). The float32 values for escape symbols are stored immediately after the rANS payload and consumed in symbol order. The complete payload is finally framed with Zstandard lossless compression.

The quantizer radius is $R = 2^{15}$, and integer CDFs use precision 24, assigning at least one count to each symbol. The stage model is selected from a pure Laplace model and 20 core-tail mixtures. A mixture combines scale-grid level l with level $\min(l + \delta, 63)$, using $\delta \in \{4, 8, 16, 24\}$ and tail weight $w \in \{0.02, 0.05, 0.12, 0.25, 0.45\}$. Selection minimizes estimated negative log probability on a regularly strided sample targeting 5% of the stage and at least 1,024 points, or all points in smaller stages. For alphabet exponents of at least 13, the search retains only the pure model and mixtures with $w = 0.02$ to limit table memory. The payload uses Zstandard level 9.

C TRAINING PROCEDURE AND HYPERPARAMETERS

C.1 OBJECTIVE AND SIMULATED RECONSTRUCTION NOISE

Training uses teacher-forced reconstructed values to expose the predictor to imperfect context, as stated in Section 2.5. A separate ε is sampled for each field, log-uniformly from 10^{-6} to 10^{-2} . The same sampled tolerance conditions every training stage and sets the perturbation amplitude. This is a surrogate for deployment, which can assign tighter tolerances to coarse stages; training does not simulate that allocation or the quantized feedback exactly. Evaluation additionally includes 10^{-1} , outside the training range.

Write $r_p = y_p - \mu_p$ for the continuous residual, so that the summand of Equation (7) integrates the Laplace density over $[r_p - \varepsilon, r_p + \varepsilon]$. Neither the residual index nor the scale-table index is rounded, so gradients reach both μ_p and β_p directly; the loss is a differentiable rate surrogate rather than the exact deployed stream length.

For large residuals, our implementation softens the Laplace tail to avoid gradients proportional to an extremely small inverse scale. When $|r_p| \geq \varepsilon$, define $t_p = (|r_p| - \varepsilon)/\beta_p$ and $t_0 = 32 \ln 2$, and replace t_p by

$$g(t_p) = \begin{cases} t_p, & t_p \leq t_0, \\ t_0 + \ln(1 + t_p - t_0), & t_p > t_0. \end{cases} \quad (10)$$

The optimized objective is then the stabilized form of Equation (7),

$$\tilde{\mathcal{L}} = \frac{1}{|\mathcal{H}|} \sum_{p \in \mathcal{H}} \begin{cases} -\log_2[F_{\text{Lap}}(r_p + \varepsilon; 0, \beta_p) - F_{\text{Lap}}(r_p - \varepsilon; 0, \beta_p)], & |r_p| < \varepsilon, \\ \frac{\ln 2 + g(t_p) - \ln(1 - \exp(-2\varepsilon/\beta_p))}{\ln 2}, & |r_p| \geq \varepsilon, \end{cases} \quad (11)$$

where the first branch floors the probability at 10^{-30} for numerical evaluation. The tail continuation preserves a mean gradient for extreme errors while reducing its magnitude. It approximates the limited usefulness of further rate penalties where deployment can escape to a raw value. Differentiation passes through the learned context and finalized embeddings within a stage sequence. Teacher-forced values do not depend on earlier mean predictions, so gradients do not pass through a simulated quantizer. Chunked training uses perturbed boundary reconstructions and discards the preceding chunk's embeddings, exposing the predictor to the deployed absence of cross-seam learned state. The classical gate, scale-grid rounding, and stage-level entropy-model selection are deployment operations and are not optimized in this training loop.

C.2 SYNTHETIC PRETRAINING

We generate fresh fields at every optimizer step using the distribution described in Section 2.5. Each step contains 16 fields of shape 128^2 and four fields of shape 16^4 , so the 2-D and 4-D fields each

Table C.1: Possible synthetic-field constructions. Each realization combines one spectrum, one value warp, and one discontinuity choice. Here $f(\mathbf{x})$ denotes the current field value at grid coordinate $\mathbf{x}$. Parameter distributions are given in the accompanying text.

Group	Realization	Formula or operation
Spectrum	Smooth	Two-scale anisotropic Gaussian filtering; see Equations (12) to (14).
	Turbulent	Radial power-law filtering followed by repeated smooth displacement; see Equations (12), (15) and (16).
Value warp	Identity	$f(\mathbf{x}) \mapsto f(\mathbf{x})$.
	Signed power	$f(\mathbf{x}) \mapsto \text{sign}(f(\mathbf{x})) f(\mathbf{x}) ^\gamma$.
	Two-phase	$f(\mathbf{x}) \mapsto \tanh(\gamma f(\mathbf{x}))$.
Jumps	None	$f(\mathbf{x}) \mapsto f(\mathbf{x})$.
	One to three hyperplanes	$f(\mathbf{x}) \mapsto f(\mathbf{x}) + \sum_{c=1}^C \Delta_c \mathbf{1}\{\boldsymbol{\nu}_c^\top \mathbf{x} > t_c\}$.

contribute 262,144 scalar values and receive equal weight in the loss. For the 2-D fields, we sample the anchor stride uniformly from $\{8, 16, 32\}$ (respectively three, four, or five refinement levels); half of these steps randomly split the field into two chunks and randomize both the split axis and chunk traversal order, exposing the model to the deployed halo exchange. The 4-D fields use stride 8 and three levels.

The construction types of Table C.1 are drawn with the continuous parameters resampled within each type, as follows.

For a field on an $n_1 \times \dots \times n_D$ grid, let $\mathbf{k} = (k_1, \dots, k_D)$ contain the discrete FFT frequencies in cycles per grid cell. We independently sample two standard-normal white-noise fields ξ_1 and ξ_2 and mirror-extend each axis to impose reflecting boundaries. Let $\mathcal{F}$ denote the D -dimensional FFT. Given amplitude filters G_1 and G_2 , the initial field is

$$f^{(0)}(\mathbf{x}) = \mathcal{F}^{-1}[G_1 \mathcal{F} \xi_1](\mathbf{x}) + \frac{1}{2} \mathcal{F}^{-1}[G_2 \mathcal{F} \xi_2](\mathbf{x}). \quad (12)$$

The realization is smooth with probability 0.5 and turbulent otherwise.

For a smooth realization, we draw a base width b log-uniformly from $[1, 32]$ cells in 2-D or $[0.5, 8]$ cells in 4-D, draw an anisotropy spread $a \sim \mathcal{U}[0, 1]$, and set

$$\sigma_j = \max(0.3, b \exp(\delta_j)), \quad \delta_j \sim \mathcal{N}(0, a^2). \quad (13)$$

The two Gaussian amplitude filters are

$$H_s(\mathbf{k}) = \exp\left(-2\pi^2 \sum_j (s\sigma_j)^2 k_j^2\right), \quad s \in \{1, 2\}. \quad (14)$$

We use $G_1 = H_1$ and $G_2 = H_2$ in Equation (12). The configured correlation inputs are $[2, 32]$ and $[1, 8]$ for 2-D and 4-D, respectively. The implementation draws b from half the lower input through the upper input, which gives the intervals above.

For a turbulent realization, we draw $p \sim \mathcal{U}[0.7, 1.6]$ and use the radial amplitude filter

$$H_{\text{turb}}(\mathbf{k}) = \begin{cases} (\|\mathbf{k}\|_2^2 + 10^{-8})^{-p/2}, & \|\mathbf{k}\|_2 > 0, \\ 0, & \|\mathbf{k}\|_2 = 0. \end{cases} \quad (15)$$

Here p is the Fourier-amplitude exponent. We use $G_1 = G_2 = H_{\text{turb}}$ in Equation (12).

The turbulent field is then advected by a smooth random displacement. Let $m = \min_j n_j$. For each component j , we Gaussian-filter an independent white-noise scalar field with isotropic width $\ell \sim \mathcal{U}[0.18m, 0.40m]$, producing a scalar field $v_j(\mathbf{x})$ that we normalize to unit spatial standard deviation. Together, these components form the vector field $\mathbf{v}(\mathbf{x}) = (v_1(\mathbf{x}), \dots, v_D(\mathbf{x}))$. A single ℓ is shared by the fields in a generated batch, while their component noise is independent. Each

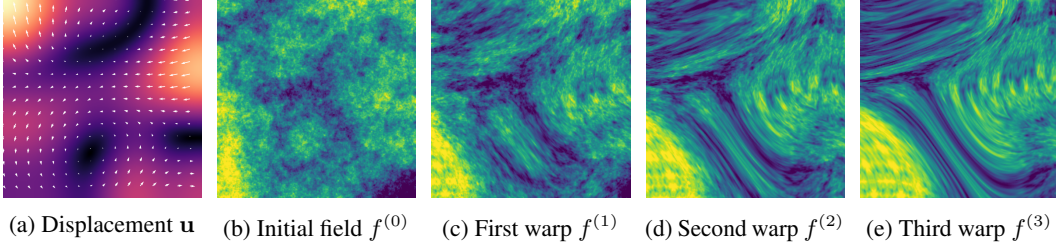


Figure C.1: Construction of a turbulent synthetic field in 2-D. A smooth random displacement $\mathbf{u}$, shown by arrows over its magnitude, repeatedly advects the initial power-law field $f^{(0)}$. Reusing the same displacement for three warps progressively bends the multiscale structure into coherent filaments.

turbulent field draws $A \sim \mathcal{U}[0.04m, 0.10m]$ and sets $\mathbf{u}(\mathbf{x}) = A\mathbf{v}(\mathbf{x})$. Starting from $f^{(0)}$, the generator applies the same frozen displacement $\mathbf{u}$ three times,

$$f^{(r+1)}(\mathbf{x}) = f^{(r)}(\mathbf{x} + \mathbf{u}(\mathbf{x})), \quad r \in \{0, 1, 2\}, \quad (16)$$

using multilinear interpolation and reflecting boundaries. Smooth fields are not spatially warped.

Figure C.1 visualizes this construction in 2-D. The same frozen displacement progressively bends the initial multiscale texture into filamentary structures.

Finally, each field f is standardized before applying one of the three value warps in Table C.1, chosen with equal probability. The signed-power exponent is $\gamma \sim \mathcal{U}[0.4, 3]$, while the two-phase exponent is $\gamma \sim \mathcal{U}[1.5, 4]$. With probability 0.25, we draw C uniformly from $\{1, 2, 3\}$ and add the hyperplane jumps shown in the table. Each normal is $\nu_c = \mathbf{g}_c / \|\mathbf{g}_c\|_2$ for $\mathbf{g}_c \sim \mathcal{N}(0, I_D)$. Its threshold t_c is the q_c -quantile of $\nu_c^\top \mathbf{x}$ over the grid, where $q_c \sim \mathcal{U}[0.25, 0.75]$. Its signed magnitude is $\Delta_c = \epsilon_c J_c s_f$, where ϵ_c is uniform on $\{-1, 1\}$, $J_c \sim \mathcal{U}[0.4, 1.5]$, and s_f is the standard deviation of f before adding any jumps. The result is mapped to $[0, 1]$ using its 1st and 99th percentiles and clipped to that interval.

The reported pretrained checkpoint has embedding width $d = 64$, aggregation level $L_{\text{agg}} = 1$, and 102,710 trainable parameters. We train it for 50,000 optimizer steps with AdamW, learning rate 5×10^{-4} , weight decay 10^{-3} , and the default $(\beta_1, \beta_2) = (0.9, 0.999)$. The learning rate increases linearly during the first 1,000 steps, remains constant through 60% of training, and then decreases linearly to zero over the final 40%. We maintain an exponential moving average of the weights with decay 0.999 and save these averaged weights. Pretraining runs on a single NVIDIA H100 and takes approximately 4 hours.

C.3 DATASET-SPECIFIC FINE-TUNING

Fine-tuning starts from the same pretrained $d = 64$, $L_{\text{agg}} = 1$ checkpoint. A batch is formed from independently located, min-max-normalized hypercubic crops. For 3-D tensors, crops have side 32 and the schedule level is sampled uniformly from $\{3, 4, 5\}$; for 4-D tensors, crops have side 16 and levels are sampled from $\{2, 3, 4\}$. One schedule depth is drawn per optimizer step. GyroSwin’s 5-D distribution-function fields use crops capped at 16 cells per axis (shorter axes are left whole) and levels $\{2, 3, 4\}$. These depth ranges deliberately train several rollout lengths instead of specializing the weights to one hierarchy. We use 10,000 steps, batch size 16 for The Well and batch size 1 for GyroSwin, AdamW with learning rate 5×10^{-5} and weight decay 10^{-3} , a 500-step linear warmup, linear decay over the final 40%, and EMA decay 0.999. The error-bound sampling and rate loss are unchanged from pretraining. Fine-tuning runs on a single NVIDIA H100 and takes approximately 1 hour for 3D, 2 hours for 4D and 4 hours for 5D. All reported evaluations run on a single NVIDIA V100.

C.4 EFFECT OF PRETRAINING DIMENSIONALITY

Adding either 3-D or 4-D synthetic fields to the 2-D pretraining distribution improves the rate of the pretrained checkpoint on every evaluation dataset. The 2-D+4-D checkpoint yields a lower BPP difference than the 2-D+3-D checkpoint on seven datasets, while the 2-D+3-D checkpoint

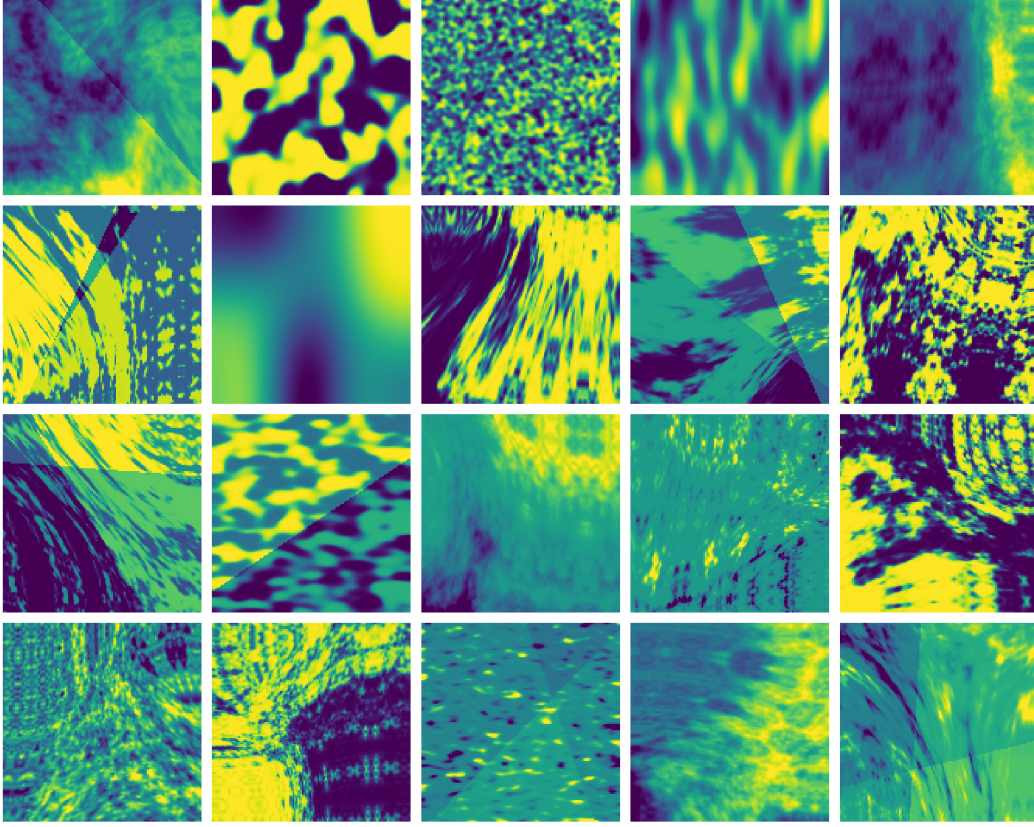


Figure C.2: Examples of 128×128 2-D fields drawn from the synthetic training distribution. The realizations span smooth and multiscale correlation, anisotropy, turbulent filaments, and sharp hyperplane jumps.

Table C.2: Effect of synthetic pretraining dimensionality on the average relative BPP difference (%; lower is better) at the same relative maximum error (RME) target, relative to SZ3. The dimensions in parentheses specify the dimensions of the synthetic fields used for pretraining, not the dimensionality supported at inference. Negative values denote a BPP saving. Blue, green, and orange column backgrounds denote 3-D, 4-D, and 5-D evaluation datasets, respectively. See Appendix D.5 for the calculation. Bold indicates the best method on each dataset, and underlined indicates the second-best.

Method	Euler-Multi-Quadrants	Gray-Scott	Hurricane ISABEL	Miranda	S3D	Convective-Envelope	Gravity-Cooling	MHD-64	RT-Instability	Supernova-Explosion	GyroSwin	Gysela	Synthetic 5D
SZ3	0	0	0	0	0	<u>0</u>	0	0	0	0	0	0	0
LATEC (2D)	28.2	9.4	-6.1	-3.3	-8.7	41.4	-2.2	5.1	-13.4	-4.7	11.1	23.9	-11.4
LATEC (2D+3D)	<u>16.9</u>	-6.8	<u>-10.9</u>	<u>-10.8</u>	<u>-13.0</u>	-2.2	-11.4	<u>-18.8</u>	-36.3	<u>-9.7</u>	3.0	-9.9	<u>-44.9</u>
LATEC (2D+4D)	18.8	<u>-4.7</u>	-14.8	-11.4	-14.5	1.9	<u>-3.4</u>	-19.0	<u>-35.9</u>	-9.9	<u>1.9</u>	<u>-3.2</u>	-47.3

is better on six. Neither checkpoint dominates simply by matching the evaluation dimensionality. Overall, the 2-D+3-D and 2-D+4-D checkpoints save BPP relative to SZ3 on eleven and ten datasets, respectively, compared with seven for 2-D-only pretraining. These results show that exposure to higher-dimensional synthetic structure is important for transfer, while the precise additional pretraining dimension has a smaller and dataset-dependent effect.

D EVALUATION PROTOCOL

D.1 EVALUATION METRICS

For a field $\mathbf{x}$ with N points and reconstruction $\hat{\mathbf{x}}$,

$$\text{RME} = \frac{\|\mathbf{x} - \hat{\mathbf{x}}\|_\infty}{\max_p x_p - \min_p x_p}, \quad \text{PSNR} = 10 \log_{10} \left(\frac{(\max_p x_p - \min_p x_p)^2}{\frac{1}{N} \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2} \right). \quad (17)$$

The range-relative maximum error directly tests the pointwise guarantee: a requested normalized bound ε corresponds to a horizontal target at ε in the curves of Figure 4.

D.2 BENCHMARK DATASETS

Table D.1: The 13 fields used in the benchmark, sorted by size.

Dataset	Field	Shape	Size (MiB)
Gysela	density	$64 \times 64 \times 64 \times 64 \times 64$	4096
Convective-Envelope	density	$64 \times 256 \times 128 \times 256$	2048
RT-Instability	density	$64 \times 128 \times 128 \times 128$	512
S3D	0	$500 \times 500 \times 500$	477
Supernova-Explosion	density	$32 \times 128 \times 128 \times 128$	256
Miranda	density	$256 \times 384 \times 384$	144
Synthetic 5D	N/A	$32 \times 32 \times 32 \times 32 \times 32$	128
Hurricane ISABEL	CLOUDf48	$100 \times 500 \times 500$	95
Euler-Multi-Quadrants	density	$64 \times 512 \times 512$	64
MHD-64	density	$64 \times 64 \times 64 \times 64$	64
GyroSwin	density	$32 \times 8 \times 16 \times 85 \times 32$	43
Gravity-Cooling	density	$32 \times 64 \times 64 \times 64$	32
Gray-Scott	A (gliders)	$256 \times 128 \times 128$	16

D.3 SYNTHETIC 5D DATASET AND BENCHMARK ADAPTERS

Synthetic 5D extends the smooth Gaussian-field family used in pretraining to five axes. The benchmark draws float32 standard Gaussian noise on a 32^5 grid with NumPy generator seed zero. It samples a base correlation length log-uniformly from $[1, 8]$, an anisotropy spread $u \sim \mathcal{U}[0, 0.8]$, and independent $z_j \sim \mathcal{N}(0, u^2)$. The smoothing widths are $\sigma_j = \max(0.5, \sigma_{\text{base}} \exp(z_j))$. A separable Gaussian filter uses reflective boundaries and truncates at three standard deviations. The field is then centered, divided by its standard deviation, multiplied by a factor drawn uniformly from $[1, 3]$, transformed with tanh, and min-max normalized to $[0, 1]$. This benchmark realization shares smoothness, anisotropy, and value-warp mechanisms with pretraining. It does not draw the turbulent component, a second smooth scale, or hyperplane jumps from the full training mixture. Its role is to test an unseen dimensionality within that construction family.

Dimension-restricted baselines receive deterministic reshapes that preserve flat element order. ZFP and HPEZ collapse leading axes into one when the tensor has more than four dimensions, retaining the last three axes. HPEZ also removes singleton axes. SPERR removes singleton axes and collapses leading axes into one depth axis when the tensor has more than three dimensions, retaining the last two axes. It codes the resulting volume in depth slabs targeting at most 2^{26} values, with boundary adjustments to avoid a singleton depth. These reshapes require no padding on the evaluated fields. SZ3 receives the native tensor shape, while TTHRESH removes singleton axes and retains the remaining dimensions. HPEZ uses its full feature configuration, $q = 4$. Pointwise tolerances are relative to the range of the original field, including when a reshaped volume is coded in slabs.

The benchmark computes BPP from the returned byte stream. For LATEC this includes the stream header, normalization extrema, gate descriptors, residual payload, and float32 escapes. Pretrained or fine-tuned model weights are supplied separately and excluded from BPP. Small dictionaries passed separately by benchmark adapters, such as shape, dtype, and checkpoint location, are also excluded.

Table D.2: CAESAR average relative BPP difference (%; lower is better) at matched quality, relative to SZ3. Negative values denote a BPP saving. Blue, green, and orange column backgrounds denote 3-D, 4-D, and 5-D datasets, respectively. Matched-RME results were interpolated because CAESAR does not provide an RME bound. ERR denotes a configuration that did not complete successfully. See Appendix D.5 for the calculation.

Metric	Method	Gray-Scott	Euler-Multi-Quadrants	Hurricane ISABEL	Miranda	S3D	Gravity-Cooling	MHD-64	Supernova-Explosion	RT-Instability	Convective-Envelope	GyroSwin	Synthetic 5D	Gysela
RME	CAESAR-D	835.2	524.2	689.2	697.0	2604.1	ERR	ERR	3250.1	748.8	6506.0	ERR	ERR	ERR
	CAESAR-V	775.6	436.7	701.5	669.2	2150.7	ERR	ERR	2774.1	715.9	5016.1	ERR	ERR	ERR
PSNR	CAESAR-D	224.2	327.0	533.7	163.0	2591.6	ERR	ERR	251.6	499.7	196.0	ERR	ERR	ERR
	CAESAR-V	239.6	262.2	494.4	148.4	1917.0	ERR	ERR	249.6	484.0	201.4	ERR	ERR	ERR

D.4 LEARNED BASELINE EVALUATION

We evaluated CAESAR because it is, to the best of our knowledge, the only other learned scientific-data compressor with a publicly available pretrained model that can be applied directly to our benchmark (Li et al., 2025a;b). CAESAR provides two pretrained spatiotemporal variants. CAESAR-V combines a variational autoencoder, scale hyperprior, and super-resolution, while CAESAR-D applies conditional latent diffusion to autoencoded keyframes. Both use a PCA residual post-process that bounds global L^2 error rather than pointwise L^∞ error directly.

Our adapter maps the last three axes to time, height, and width, merges preceding axes into channels, and uses a batch axis of one. The decoder removes temporal padding and restores the original shape. This fixed interpretation is an adapter convention, not a claim that every field’s axes represent those modalities. Both variants failed to complete on five higher-dimensional fields and required substantially more BPP than SZ3 wherever they completed, under both matched-RME and matched-PSNR evaluation. Table D.2 reports the complete results.

The error targets in this benchmark are far from CAESAR’s intended operating regime. CAESAR is designed for very high compression ratios, where the autoencoded latent carries the field and the PCA residual stage only refines it. At our tighter targets, the latent is nearly a fixed cost while the residual correction supplies almost all of the accuracy, so most transmitted bits belong to the PCA residual stream, and results show that CAESAR is poorly matched to our evaluation regime.

D.5 COMPUTING THE RATE GAINS IN THE TABLES

Tables 2 and E.1 summarize the rate-quality curves as average relative BPP differences with respect to SZ3. At each matched quality level, with SZ3 rate r_{SZ3} and compared-method rate r_m , we compute

$$g_m = \frac{r_m}{r_{\text{SZ3}}} - 1. \quad (18)$$

We then average g_m over the matched levels. Negative values therefore mean that the method uses fewer bits per point than SZ3 at matched quality, whereas positive values mean that it uses more.

For both tables, we evaluate the compressors at logarithmically spaced RME targets from 10^{-1} to 10^{-6} , except on GyroSwin, where we stop at 10^{-5} because its values are stored in bfloat16. For the maximum-error table, the matched levels are the error bounds configured for SZ3. For compressors that are also parameterized by an error bound, we retain only bounds run by both methods and use the median BPP over repeated runs at each bound. TTHRESH has no configured RME bound, so we estimate its BPP at each SZ3 bound from its achieved-error curve by log-log interpolation of BPP against achieved range-relative maximum error; targets outside its measured range are omitted. This is why its entries are marked as estimates. We apply the same procedure to the CAESAR results in Table D.2.

For the PSNR table, we first form each method’s curve from the median (PSNR, BPP) of each parameter configuration. For a dataset, we identify the PSNR interval covered by every solver

Table E.1: Average percentage BPP difference from SZ3 across matched PSNR levels, so negative values are rate savings and lower is better; Appendix D.5 gives the calculation. Blue, green, and orange backgrounds mark 3-D, 4-D, and 5-D datasets. A slash marks a dataset with no separate training split, hence no fine-tuned checkpoint. Bold is best, underlined second-best.

Method	Euler-Multi-Quadrants	Gray-Scott	Hurricane ISABEL	Miranda	S3D	Convective-Envelope	Gravity-Cooling	MHD-64	RT-Instability	Supernova-Explosion	GyroSwin	Gysela	Synthetic 5D
SZ3	0	0	<u>0</u>	0	0	<u>0</u>	0	0	0	0	<u>0</u>	0	0
ZFP	129.5	38.8	50.2	111.2	873.6	37.0	22.4	11.3	13.6	83.8	20.9	37.5	2.8
SPERR	<u>-3.2</u>	-33.3	113.8	<u>-47.9</u>	<u>-53.7</u>	73.4	4.4	-9.7	16.5	115.1	2.8	<u>-35.9</u>	-1.8
HPEZ	-37.5	-4.2	19.6	-24.5	-27.7	36.6	3.2	1.4	10.8	72.4	2.1	23.1	4.2
TTHRESH	176.3	<u>-20.3</u>	399.6	-66.5	-66.3	-4.0	47.3	-11.4	139.1	43.4	0.9	-91.9	<u>-20.3</u>
LATEC	24.0	11.0	-10.2	-7.5	17.6	3.5	<u>-6.1</u>	<u>-14.8</u>	<u>-27.0</u>	<u>-2.2</u>	1.4	15.4	-39.8
LATEC (fine-tuned)	16.8	-9.0	/	/	/	0.5	-17.3	-17.3	-31.4	-12.1	-0.8	/	/

with at least two valid points, sample it uniformly at 100 values, and estimate each BPP by linear interpolation in PSNR and logarithmic interpolation in BPP. The reported value is the mean of g_m over these 100 common-PSNR samples. No extrapolation is performed in either table. The RME averages can therefore contain different target subsets for different methods, whereas the PSNR comparison uses one common interval per field. The latter interval depends on the methods with valid curves.

E ADDITIONAL EXPERIMENTAL ANALYSES

E.1 RATE-DISTORTION AT MATCHED PSNR

Table E.1 is the matched-PSNR counterpart of Table 2, computed as described in Appendix D.5. It measures average squared error at a given rate rather than the pointwise bound this work targets, and is reported here for completeness; the body discusses why the two views differ.

E.2 ABLATION STUDIES

We examine how the two learned outputs, the GNN mean and scale, contribute to compression. The mean provides a data-adaptive prediction that complements classical interpolation. The scale serves both as a spatially varying entropy parameter and as the pointwise signal used to gate between these predictions. Figures E.1 and E.3 report component ablations on Euler-Multi-Quadrants, Miranda, and MHD-64, together with an aggregate over 11 datasets, at relative error bounds from 10^{-1} to 10^{-5} . Stream sizes are normalized by the full LATEC configuration at the same bound, so a ratio above one indicates a larger stream. Figure E.5 complements these rate comparisons with the fractions of points assigned to each predictor family.

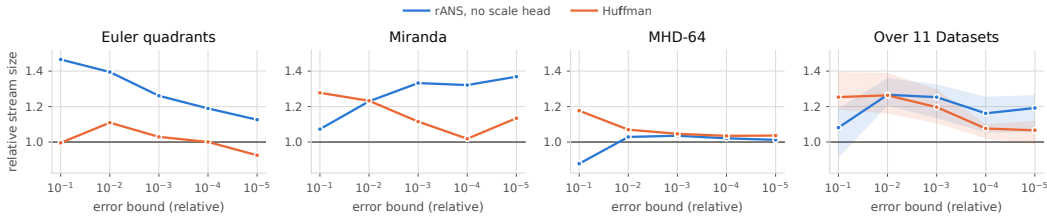


Figure E.1: Ablations of scale estimation and entropy coding. In the rightmost panel, each line is the mean over 11 datasets and its shaded band spans the 25th to 75th percentiles.

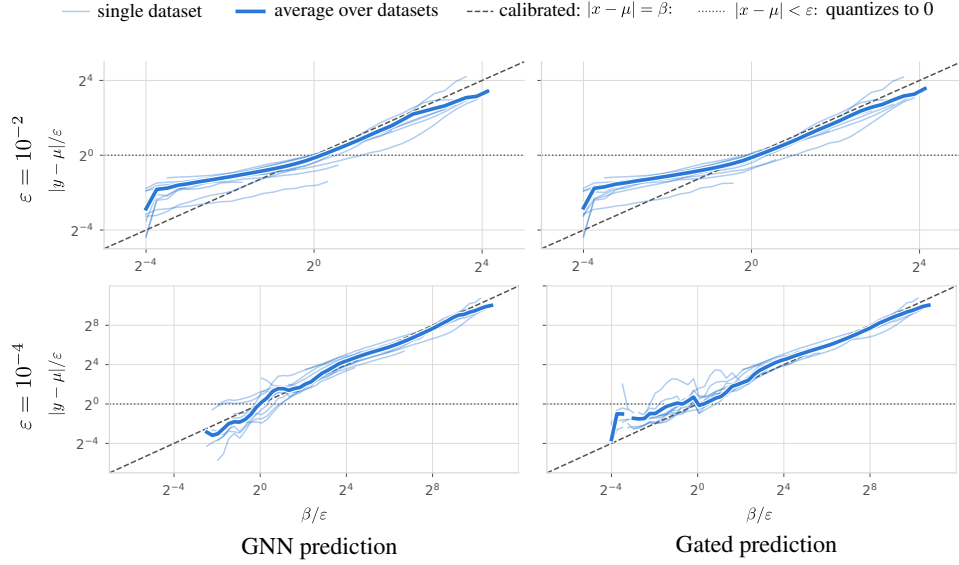


Figure E.2: Consistency between the predicted Laplace scale and empirical residual magnitude for the raw GNN mean and the gated mean at relative error bounds 10^{-2} and 10^{-4} . At each predicted scale, a curve reports the mean absolute residual of the points assigned that scale. Each light curve corresponds to one dataset, and the dark curve averages across datasets. The dashed diagonal shows the moment relation $\mathbb{E}|y_p - \hat{\mu}_p| = \hat{\beta}_p$ implied by a zero-centered Laplace model. The dotted horizontal line is the boundary $|y_p - \hat{\mu}_p| = \varepsilon_i$, below which a residual quantizes to the zero index.

E.2.1 SCALE ESTIMATION AND ENTROPY CODING

The learned scale improves compression overall (Figure E.1). Across the 11-dataset aggregate, removing the scale head increases mean stream size at every bound. The field-level panels show that the gain is not uniform: the scale is consistently valuable on Euler-Multi-Quadrants and Miranda, while on MHD-64 it has little effect at tighter bounds and is detrimental at the loosest one. The aggregate trend nevertheless shows that adapting the residual distribution point by point is preferable to rANS without the learned pointwise scale. Replacing rANS with Huffman coding also increases mean stream size at every bound, confirming that the learned scale is useful when coupled to a coder that can exploit its probability estimates.

The scale $\hat{\beta}_p$ is not trained to estimate the individual magnitude $|y_p - \hat{\mu}_p|$. It parameterizes the conditional Laplace distribution from which the entropy model assigns residual probabilities. This distribution nevertheless implies the group-level moment $\mathbb{E}|y_p - \hat{\mu}_p| = \hat{\beta}_p$. Equivalently, for residuals sharing one scale, their Laplace negative log-likelihood is minimized when that scale equals their mean absolute magnitude. We therefore group points by predicted scale in Figure E.2 and compare this empirical mean with the scale itself. The comparison tests one consequence of the assumed distribution, rather than pointwise residual accuracy or full distributional calibration. It is relevant to coding because a scale that is systematically too small or too large assigns an unnecessarily narrow or broad residual distribution and increases expected code length.

Both the raw GNN and gated scales satisfy this moment relation approximately over most of the represented range. At the tighter 10^{-4} bound, the dataset-level curves broadly follow the diagonal across several scale levels. At the looser 10^{-2} bound, they flatten at small scales and lie mostly below the boundary marking the zero-index region. Scale mismatch is less consequential there because deviations that do not move a residual across a quantization boundary leave its coded symbol unchanged. Gating changes the low-scale behavior more than the high-scale behavior, yet preserves the overall ordering between scale and empirical residual magnitude. The scale can therefore support both predictor selection and entropy coding without estimating each residual or matching the full residual distribution exactly.

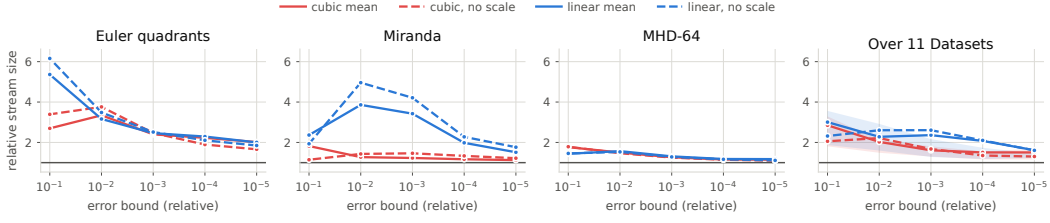


Figure E.3: Ablations of the prediction mean and its interaction with scale estimation. In the rightmost panel, each line is the mean over 11 datasets and its shaded band spans the 25th to 75th percentiles.

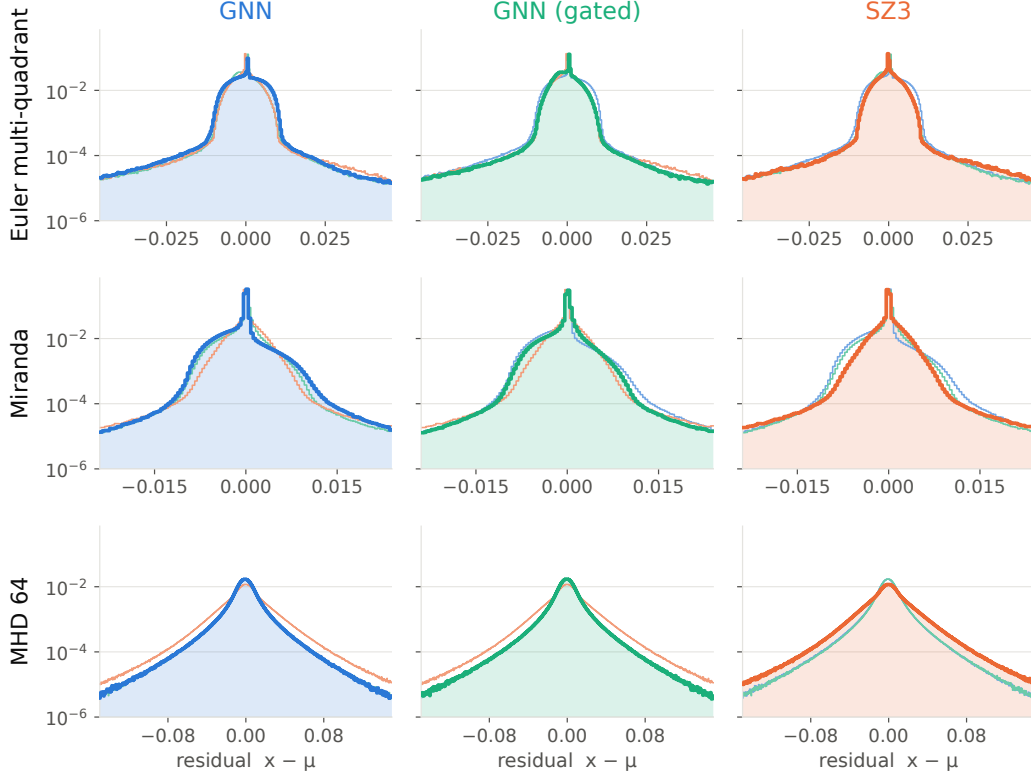


Figure E.4: Distributions of prediction residuals $y_p - \mu_p$ at $\varepsilon = 0.01$ for Euler-Multi-Quadrants, Miranda, and MHD-64. Columns compare the raw GNN, GNN with gating, and SZ3.

E.2.2 PREDICTION AND SCALE MUST BE CONSIDERED TOGETHER

The GNN mean is important because no fixed interpolation mean matches the full adaptive predictor across datasets (Figure E.3). Replacing it and its gate with either cubic or linear interpolation increases the mean stream size at every tested bound. Cubic interpolation is consistently the stronger fixed alternative in the 11-dataset aggregate, yet it remains more costly than the complete predictor. Retaining the learned scale usually reduces the cost of both fixed means, although the ordering can reverse at individual operating points. The gain therefore comes from the combination of a learned mean, classical fallbacks, and a scale model matched to the resulting residuals, rather than from one universally best prediction rule.

The residual distributions in Figure E.4 show how prediction errors differ across the same three fields. At a relative error bound of 10^{-2} , all predictors concentrate residuals near zero, but their tails and structures vary by field.

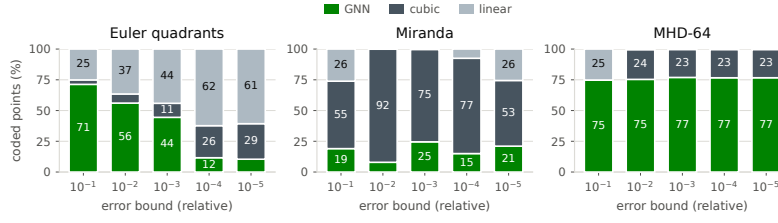


Figure E.5: Fraction of coded points assigned by the full gate to the GNN, cubic interpolation, and linear interpolation.

E.2.3 ADAPTIVE PREDICTOR SELECTION

The learned scale also enables fine-grained selection of the prediction mean at low signaling cost (Figure E.5). Although the encoder transmits only one threshold rule per chunk-stage, the learned scale β_p varies across points, so applying that rule produces a pointwise gate without transmitting pointwise flags. MHD-64 assigns about three quarters of its points to the GNN throughout the tested range, with nearly all remaining points assigned to linear interpolation. Miranda instead relies mainly on cubic interpolation, while Euler-Multi-Quadrants shifts from 71% GNN selection at 10^{-1} toward predominantly linear interpolation at the tightest bounds. These distinct selections show that the GNN mean is highly useful but not uniformly optimal. The scale-guided gate retains it where it is competitive and substitutes a classical mean elsewhere, combining local adaptivity with one compact descriptor and one global scale shift per chunk-stage.

Figure E.6 breaks the selected gates down by their remaining parameters: the threshold exponent T , the direction d , and the scale shift s . The selected threshold moves toward larger exponents as the bound tightens, most visibly on MHD-64, since the scales β_p grow relative to ε_i when the bound tightens. Miranda almost always selects $d = +1$, which replaces the GNN mean on the low-scale points, whereas MHD-64 uses both directions in similar proportions. The shift is mostly $s \in \{0, 2\}$, with $s = 4$ appearing on Euler-Multi-Quadrants at 10^{-4} and $s = 6$ remaining rare, so the scale correction is usually moderate.

E.3 RUNTIME AND MEMORY MEASUREMENTS

All throughput and memory measurements were collected on an NVIDIA V100 GPU with 32 GiB of memory and an Intel(R) Xeon(R) CPU E5-2698 v4 system with 20 CPU cores. Table 3 reports encoding and decoding throughput and peak host memory across dimensionalities. Figure E.7 shows encoding time, decoding time, peak host RSS, and peak GPU VRAM as functions of input size on 4-D tensors, together with their linear fits. The time fits provide throughput and fixed runtime overheads, while the memory fits provide memory growth per unit of input and fixed memory overheads. The fixed overhead gives LATEC a larger relative footprint on the smallest tensors, but these already fit readily in host memory; scaling matters for the tensors of up to 4 GiB that we compress in practice. SZ3 is faster by approximately 12-24 times for encoding and 13-27 times for decoding across the evaluated dimensionalities.

E.4 PREDICTED FIELDS, SCALES, AND RESIDUALS

Figure E.8 visualizes what the predictor produces on a 2-D slice of four benchmark fields at relative error bound 10^{-4} . Each row shows one quantity: the ground-truth field, the mean, the quantized residual normalized by the scale, and the scale itself, the last three given both for the raw GNN head and for the gated predictor the codec actually uses. Means and residuals use a diverging blue-to-red map from negative to positive, and the scale a logarithmic map from dark to bright. Panels are scaled independently, except the two residual rows, which share one color scale per dataset so that the GNN and gated residuals can be compared directly.

The predicted mean reproduces the large-scale structure of each field, while the predicted scale concentrates on the interfaces, filaments, and shocks where the interpolation is least reliable, and stays near zero in smooth regions. The normalized residuals are correspondingly largest where the

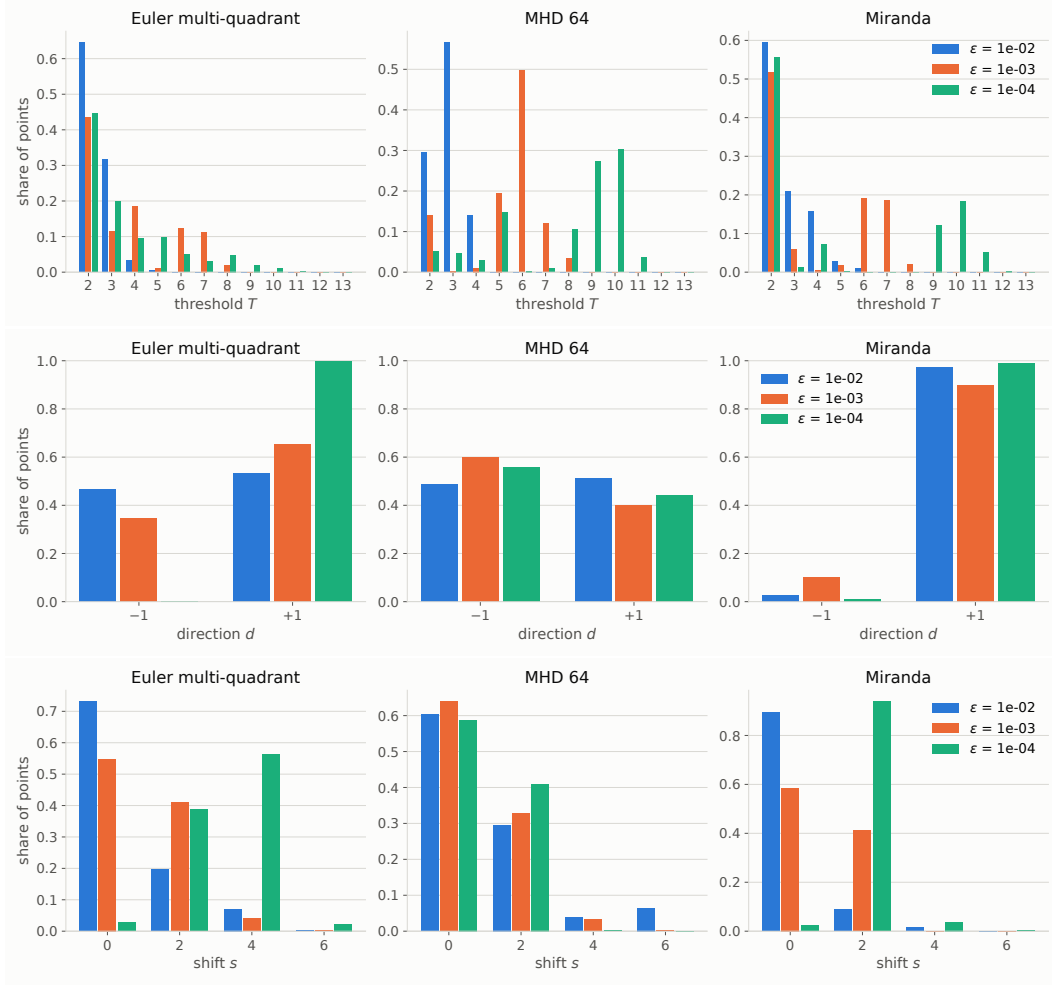


Figure E.6: Distribution of the selected gate parameters of Equation (6) over gated points (top to bottom: threshold exponent T , direction d , scale shift s), for Euler-Multi-Quadrants, MHD-64, and Miranda at $\varepsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}$.

scale is largest, which is the behavior the entropy coder relies on: a spatially varying scale that tracks residual magnitude is what turns the residual field into a cheap stream. The GNN and gated rows agree on MHD-64 and RT-Instability, where the gate selects the GNN almost everywhere, and differ on Gray-Scott and S3D, where interpolation is selected for most points.

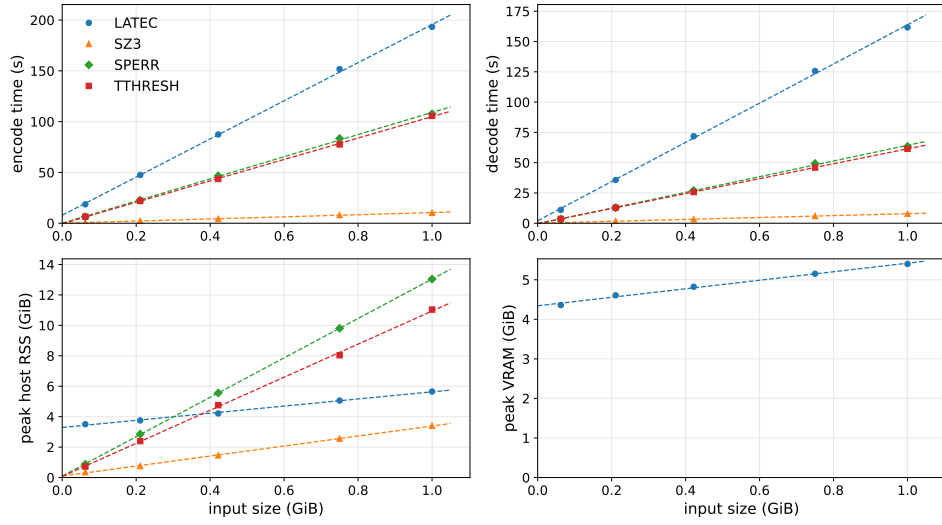


Figure E.7: Runtime and memory scaling on 4-D tensors at a relative error bound of 10^{-4} . The top row shows encoding time on the left and decoding time on the right; the bottom row shows peak host RSS on the left and peak GPU VRAM on the right. GPU memory is reported for LATEC only. Markers show measurements, and dashed lines show affine fits as a function of input size.

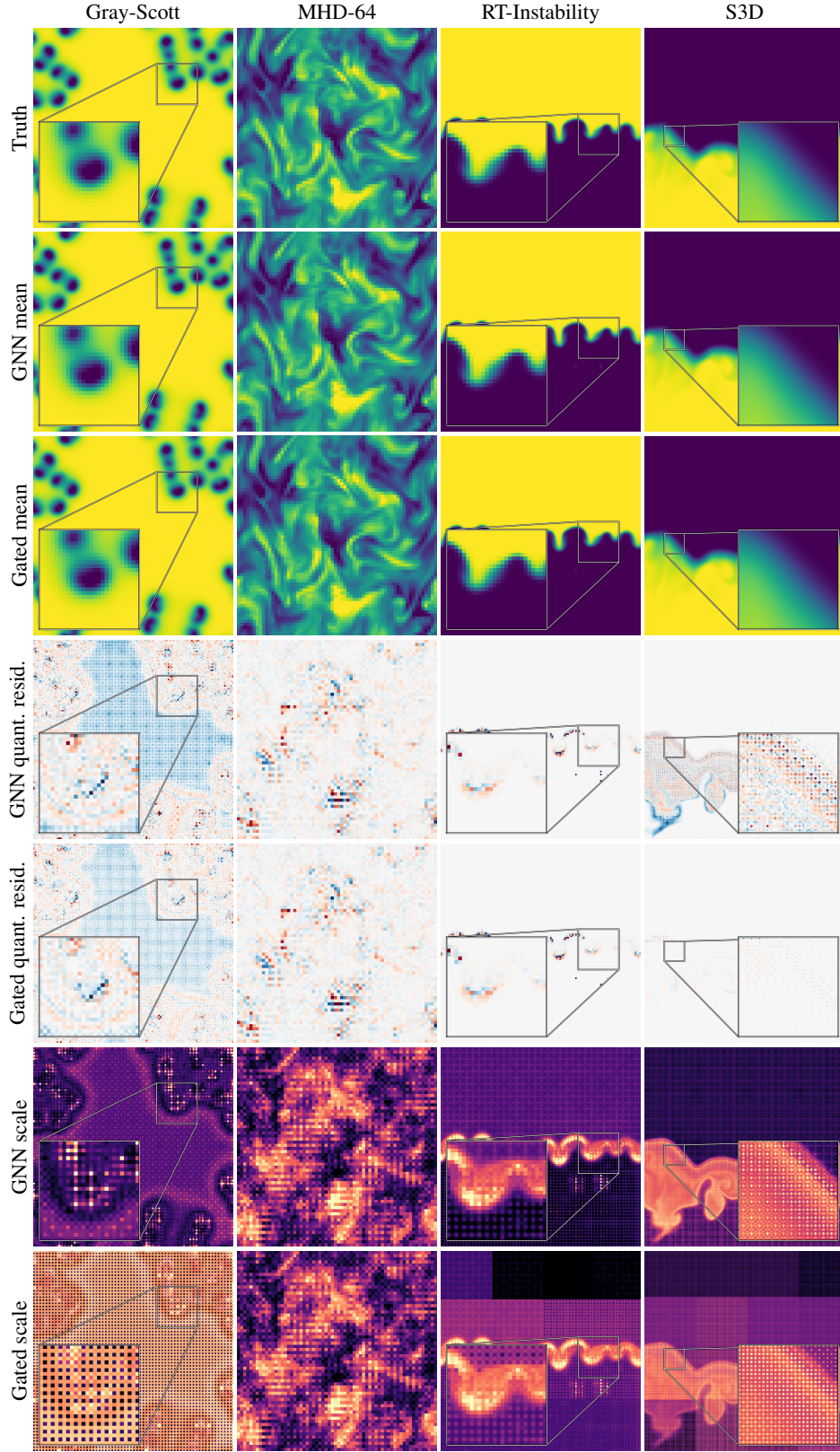


Figure E.8: Predictor outputs on a 2-D slice of four benchmark fields at relative error bound 10^{-4} , for the raw GNN head and for the gated predictor.